

Since many applications were written during the '80s on machines having only real mode, DOS 5.0²⁰ kept running that way and as a result its routines and system calls were incompatible with protected mode. This created an awkward situation where the de-facto operating system delivered with every machine sold prevented programmers from using the machine at its **full potential**. Developers were forced to ignore all the features of a **1992 CPU** and instead use it like a very fast **Intel 8086 CPU** from **1976**. They were thus limited to the following characteristics:

- ALU
- 16 registers:
 - 16-bit General Purpose Registers: AX, BX, CX, DX
 - 16-bit Index Registers: SI, DI, BP, SP
 - 16-bit Program Counter: IP
 - 16-bit Segment Registers: CS, DS, ES, FS, GS, SS
 - 16-bit Status Register
- Up to 1MiB of RAM

ote-

Trivia : One year earlier, in **1991**, a student from the **University of Helsinki** started working on a hobby ("nothing big") of his: an operating system which contrary to **DOS** was able to use the **CPU in protected mode** and take advantage of the **MMU** and the **32-bit registers**. It would become Microsoft's worst **nightmare**. **Linus Torvalds** had just started what would become **Linux**.

2.2.2 The Infamous Real Mode: 1MiB RAM limit

largura

With protected mode **unavailable**, **1991** developers programmed like it was **1976**: with a 20-bit-wide address bus offering only 1MiB of addressable RAM. Regardless how much memory was installed on the machine, only 1MiB could be addressed. To top it all off, addressing had to be done not with the 32-bit registers available but by combining two 16-bit registers. One was the segment, the other an offset within that segment. Hence the name: '16-bit segmented programming'.

The memory layout is as follows:

- From 00000h to 003FFh : the Interrupt Vector Table.
- From 00400h to 004FFh : BIOS data.
- From 00500h to 005FFh : command.com+io.sys.

²⁰Released in June 1991.

- From 00600h to 9FFFFh : Usable by a program (about 620KiB in the best case).
- From A0000h to FFFFFh : UMA (Upper Memory Area): Reserved to BIOS ROM, video card and sound card mapped I/O.

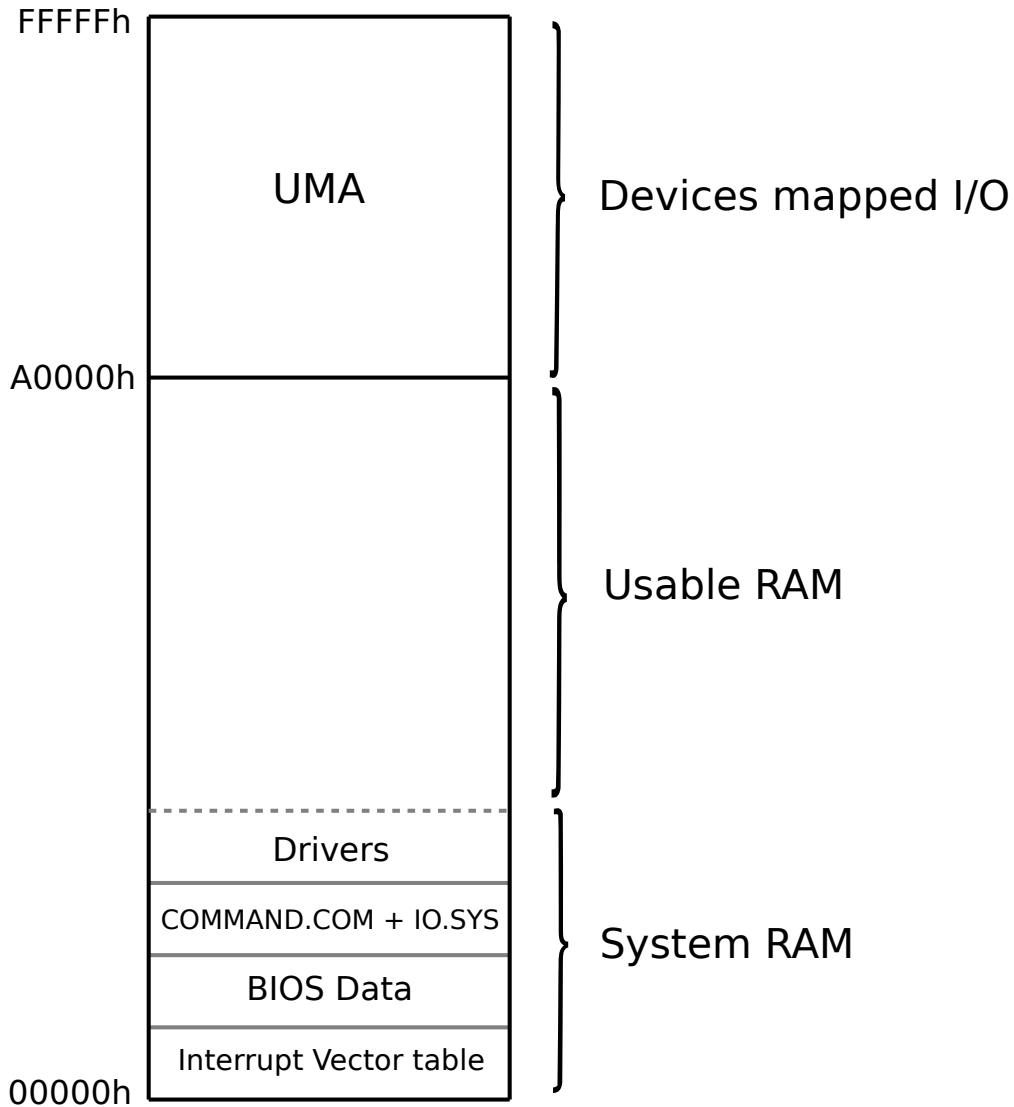


Figure 2.15: First 1MiB of RAM layout.

Out of the original 1024KiB, only 640KiB (called Conventional Memory) was accessible to a program. 384KiB was reserved for the UMA and every single driver installed (.SYS and .COM) took away from the remaining 640KiB.

Trivia : In France, people had to load KEYBFR.SYS driver so AZERTY keyboard keys would be properly mapped. The driver consumed a whopping 5KiB of Conventional Memory. Needless to say French people learned pretty quickly that god mode was IDDAD²¹.

2.2.3 The Infamous Real Mode: 16-bit Segmented addressing

With a 20-bit address bus and registers too small to contain a whole address (16-bit wide), Intel had to come up with an addressing system. Their solution was to combine two 16-bit registers, one designating a segment and the other an offset within that segment.

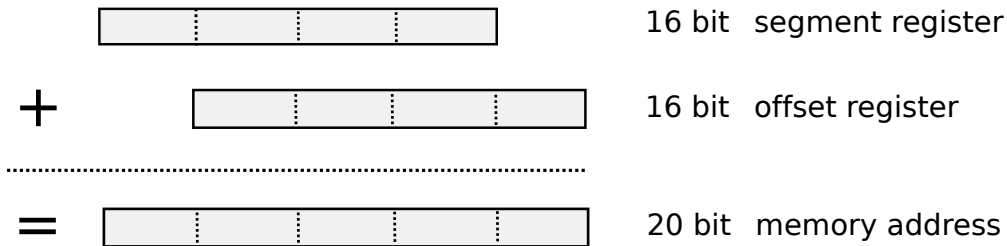


Figure 2.16: How registers are combined to address memory.

There are two kinds of pointers: near and far. A near pointer is 16 bits and considered *fast* because it can be used as is (but it only allows a `jmp` in the current code segment). A far pointer can access *anything* and allows a `jmp anywhere` but is *slower* since a 16-bit segment register has to be shifted left 4 bits and combined with the other 16-bit-offset register to form a 20-bit address.

That may not sound too bad, but in practice this segmented addressing leads to many issues. The least problematic is about the language. Since C was invented on a flat memory machine, it had to be augmented by PC compiler manufacturers. That is how the near and far keywords came into existence. Macro `MK_FP` built them and `FP_SEG/FP_OFF` accessed individual components. `libc` is also "different": `malloc` returns a near pointer and therefore can only allocate up to 64KiB. To get more than 64KiB, `farmalloc` is needed.

The larger issue is that two pointers referring to the same address can fail an equality test. In this model, the 1MiB of RAM is divided in 65536 paragraphs by the segment pointer.

²¹Invincibility mode in `Doom` is `IDDQD` on a qwerty keyboard but `IDDAD` on an azerty keyboard without the French driver loaded.

A paragraph is 16 bytes but an offset can be up to 65536 bytes which results in many overlaps. This can be explained with the following examples.

Pointer A defined as:

0000 0000 0000 0000	Segment	16 bits
+ 0000 0001 0010 0000	Offset	16 bits
=====		
0000 0000 0001 0010 0000	Address	20 bits

Pointer B defined as:

0000 0000 0001 0000	Segment	16 bits
+ 0000 0000 0010 0000	Offset	16 bits
=====		
0000 0000 0001 0010 0000	Address	20 bits

Pointer C defined as:

0000 0000 0001 0010	Segment	16 bits
+ 0000 0000 0000 0000	Offset	16 bits
=====		
0000 0000 0001 0010 0000	Address	20 bits

As defined, A, B, and C all point to the same memory location however they will fail a comparison test.

```
#include <stdio.h>
#include <dos.h>

int main(int argc, char** argv){

    void far *a = MK_FP(0x0000, 0x0120);
    void far *b = MK_FP(0x0010, 0x0020);
    void far *c = MK_FP(0x0012, 0x0000);

    printf("%d\n", a==b);
    printf("%d\n", a==c);
    printf("%d\n", b==c);
}
```

Will output:

```
0
0
0
```

With this system, pointer arithmetic must also receive careful consideration. A `far` pointer increment only increments the offset, not the segment. If you iterate on an array larger than 64KiB you will end up wrapping around. You could use yet another type of pointer `int huge*` to make pointer arithmetic work beyond 64KiB but really, nobody wants to go there.

2.2.4 Extended Memory

The 20-bit address bus of real mode limits the addressable RAM to 1MiB. Machines of 1992 came equipped with more, typically 2MiB and even sometimes 4MiB for the most fortunate customers. This memory located beyond the addressable space is called "Extended Memory". The workaround at the time to access these resources was to install specialized drivers²².

Unfortunately extended memory access was not standardized. Users could load either of the drivers provided with DOS:

- Expanded Memory Specification (EMS) drivers: EMM386.EXE.
- eXtended Memory Specification (XMS) drivers: HIMEM.SYS.

Or they could have no idea they had to install a driver, load nothing at startup, and not use any of the RAM beyond 1MiB. This use case was a big issue. Many customers could not understand why despite installing (extremely expensive²³) megabytes of RAM on their machine, the game they just purchased would refuse to start up claiming "Not enough memory". id Software had to publish an explanation (see Appendix "The 640KB Barrier") along with the game.

The two APIs had similar features but completely different architecture.

2.2.4.1 XMS API

The XMS driver works²⁴ like `malloc/free` from `libc` and is the most intuitive to a programmer. It allows manipulation of data in the non-addressable extended memory via operations such as `allocate`, `free`, `realloc`, and `move`. The key aspect of XMS is that memory has to be copied between extended RAM and conventional RAM.

²²See CONFIG.SYS in the Appendices.

²³In 1992, 4MiB of RAM cost \$149 which adjusted to inflation would be \$256 in 2017.

²⁴eXtended Memory Specification (XMS) July 19, 1988.

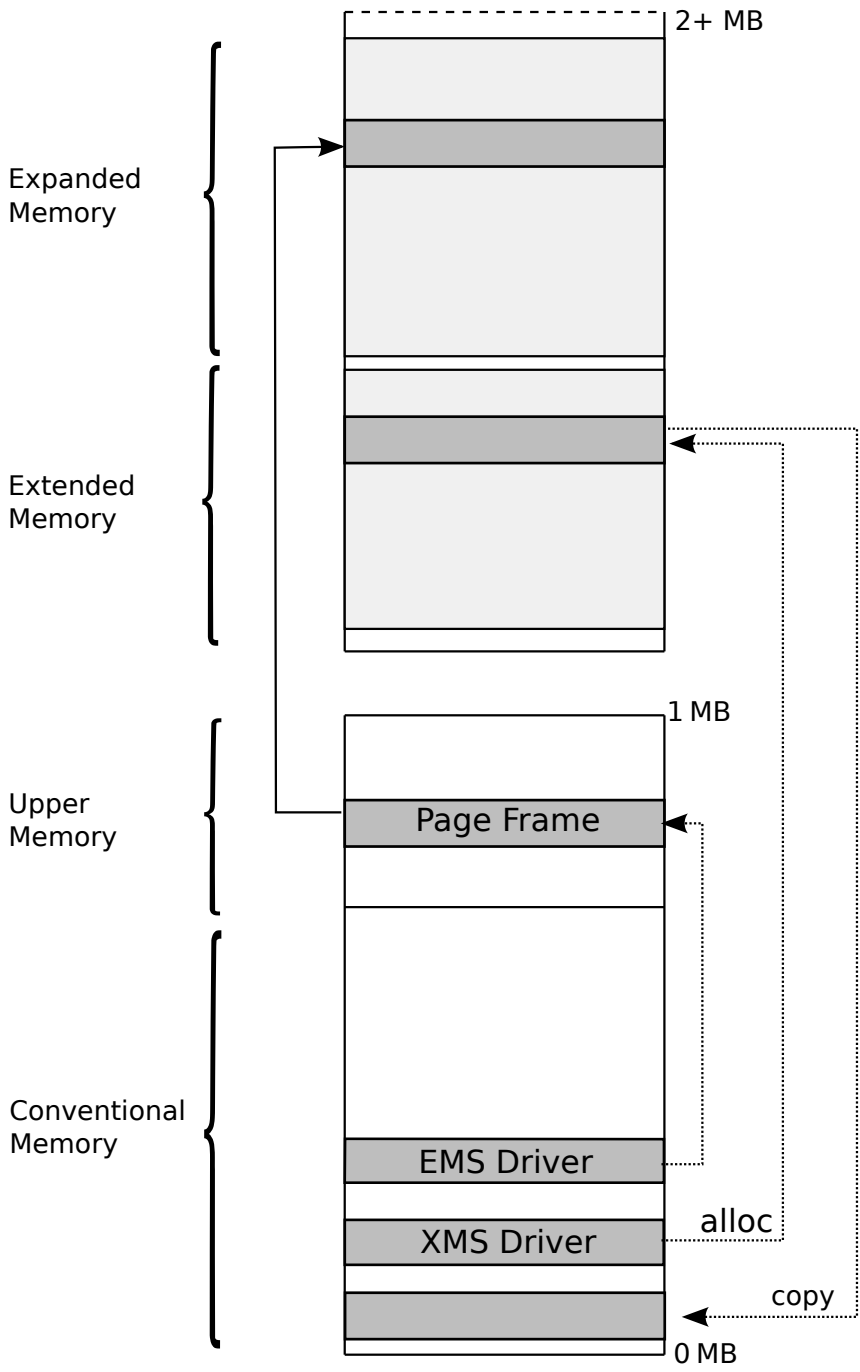


Figure 2.17: Expanded/Extended memory layout

2.2.4.2 EMS API

The EMS driver opens a window beyond the addressable RAM, as shown on page 45. The idea revolves around memory mapping. The driver allows for manipulating four units of 16KiB called "pages" via a 64KiB area called "Page Frame". Upon request to the driver, a page can be swapped into the page frame without copying anything.

2.2.4.3 EMS vs XMS

How the drivers achieved the impossible (after all how do you access RAM past 1 MiB with 20-bit pointer?) and what were the trade-offs of each approach is a fascinating topic explained in detail in Appendix B on page 297.

If you prefer not to dive into too much detail, just remember that the EMS mapping approach was several times faster than the XMS copying approach. This speed consideration considerably impacted the memory management of Wolfenstein 3D.

2.2.4.4 A system "impossible to love"

At this point, if you are puzzled by the CPU and its design you are not alone. Over the years I came across many ways to describe this madness but three particularly stand out.

“

The x86 is an architecture that is difficult to explain and impossible to love.

David Patterson & John Hennessy - Computer Organization and Design.

”

“

That sounds odd, but Intel built it, Microsoft wrote it, and DOS grew up around it.

Eccles-Jordan Trigger - Codeproject.com.

”

“

Software poison.

Steve Morris - Co-Architect of the Intel 8086.

”

Trivia : 640KiB was all a game could have for executable code. But people writing compilers got clever. Strike Commander (a famous flight arcade game released in 1993) exe-

cutable is 745KiB, which obviously doesn't fit in Conventional Memory. The trick is to use a technique of "overlay" pages where only a few overlays are loaded when the game starts. When the CPU is about to reach the end of an overlay, special instructions (inserted by the compiler) load the next overlays from HDD and setup a `jmp` instruction for the CPU to follow. The technique can be seen as an exercise in graph paging similar to how Gromit places rails in front of an ongoing miniature train as he is sitting on top of it in "The Wrong Trousers".

Trivia : As of 2017, more than forty five years after the introduction of the 8086, in the name of backward compatibility, all PCs in the world still start in real mode. A bootloader switches them to protected mode, loads the kernel, and then actual startup can begin.

2.3 Video

PCs were connected to CRT monitors: big, heavy, small diagonal, cathode ray-based, curved-surface screens. Most had a 14" diagonal with a 4:3 aspect ratio.

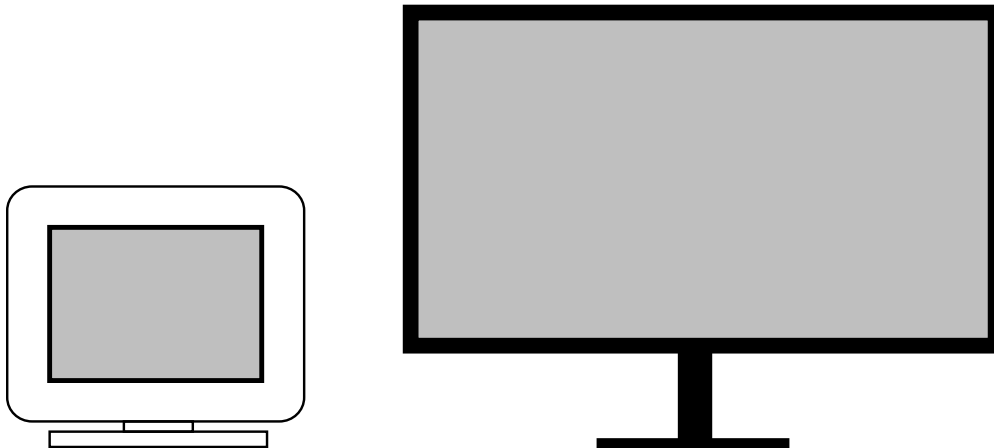


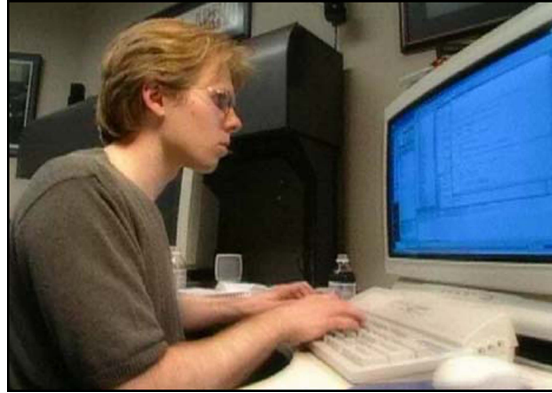
Figure 2.18: CRT (left) vs LCD (right)

To give you an idea of the size and resolution, figure 2.18 shows a comparison between a 14" CRT from 1992 (capable of a resolution of 640x480) and a 30" Apple Cinema Display from 2014 (capable of a resolution of 2560x1600).

Trivia : Despite their difference of capabilities, both monitors are the same weight: 27.5 pounds (12 kg).

Trivia : How big and heavy could a CRT be? The InterView 28hd96 by Integraph had a 28" diagonal allowing a resolution of 1920x1080 at a time where most monitors displayed 640x480. It weighed 45kg (99.5lb). For comparison, a modern DELL LCD 27" weighs 7.8kg (17lb).

In the photo on the right : John Carmack in 1996 working on a 28hd96 while programming Quake 2 using Visual Studio C++.



The main issue with this system was that CRTs were analog systems while computers were digital. An interface was needed between the two and it came as a series of chipsets called "Adapters".

2.3.1 History of Video Adapters

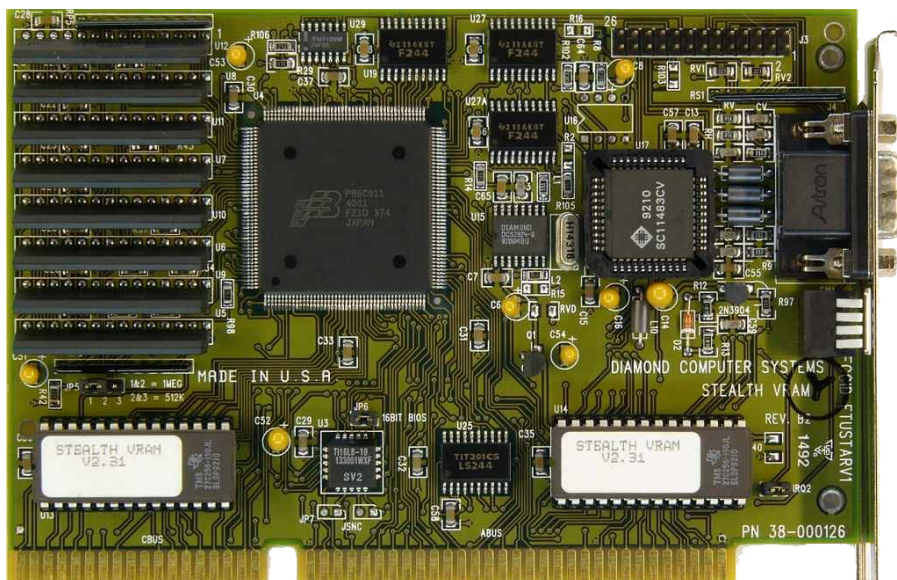
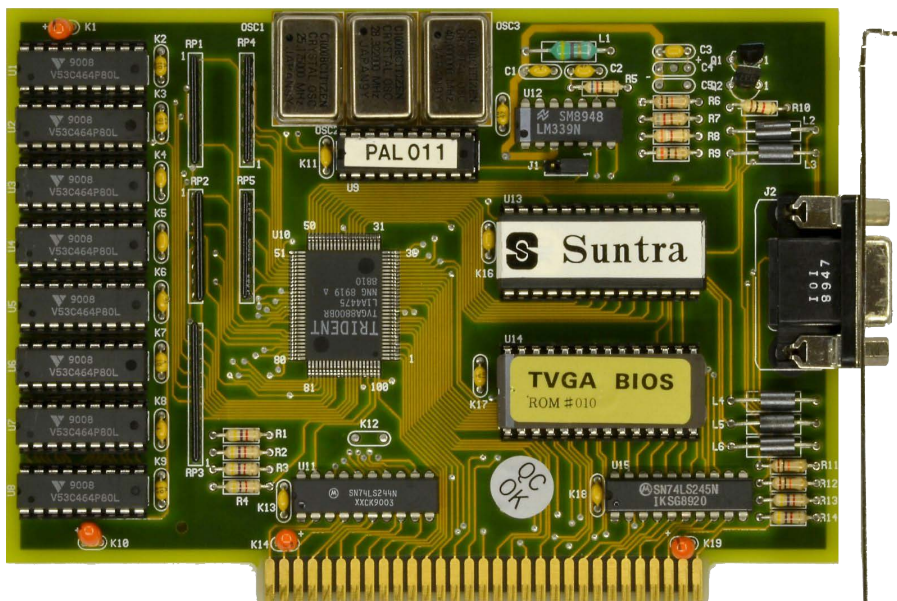
The Monochrome Display Adapter (MDA) was released in 1981 with the IBM PC 5150. It offered two colors, allowing 80 columns by 25 lines of text. While not great, it was standard on every PC. Many other systems followed over the years, each of them preserving backward compatibility.

Name	Year Released
MDA (M onochrome D isplay A dapter)	1981
CGA (C olor G raphics A dapter)	1981
EGA (E nhanced G raphics A dapter)	1985
VGA (V ideo G raphics A rray)	1987

Figure 2.19: Video interface history.

Each iteration added new features and by 1991 the predominant graphic system was VGA. All video cards installed on PCs had to follow the standard set by IBM. The universality of that system was a double-edged sword. While developers had to program for only one graphic system, there was no escaping its shortcomings.

Opposite page, on top, the VGA card Trident 8800 (8-bit ISA). The eight chips on the left of the card form the VRAM where the **framebuffers** are stored²⁵. On the bottom, a Diamond Stealth (16-bit ISA). Both photos are courtesy of vgamuseum.info.



²⁵This Trident TVGA8800BR is actually more than VGA capable since its eight V53C464P80L chips can store 64KiB each, accounting for a total of 512KiB VRAM. Cards featuring more than 256KiB were called Super-VGA but this is another story altogether.

Trivia : There was no GPU market back then. Since all video cards "only" had to be VGA compatible with 256KiB of RAM, many just bought the cheapest thing available. Note however that some cards had an 8-bit-ISA bus connector (like the Trident) and some had a 16-bit ISA connector (like the Diamond) which made them almost twice as fast.

2.3.2 VGA Architecture

VGA can be summarized as three major systems made of input, storage, and output:

- The Graphic Controller and Sequence Controller controlling how VGA RAM is accessed (the CPU-VRAM interface)
- The framebuffer (the VRAM) made of four memory banks of 64KiB (rather than one bank of 256KiB)
- The CRT Controller and the DAC²⁶ taking care of converting the palette-indexed framebuffer to RGB and then to analog signal (VRAM-CRT interface)

The most surprising part of the architecture is obviously the framebuffer. Why have four small fragmented banks instead of one big linear one?

Part of the explanation comes from backward compatibility. EGA, the predecessor of VGA had only 64KiB of RAM. It was very easy to design a backward compatible system that used only one bank of 64KiB.

The more significant reason was RAM latency and the need for minimum bandwidth. A CRT running at 60Hz and displaying 640x480 in 16 colors needs a pixel every $\frac{1}{640 \times 480 \times 60}$ th of a second. At this resolution, one pixel is encoded with 4 bits. Each nibble is translated to a 666 RGB color by the DAC. That encoding divides bandwidth by 4.5, but still requires one byte every 108 nano-seconds.

Unfortunately, RAM access latency was 200ns - not nearly fast enough²⁷ to refresh the screen at 60hz, so the DAC would starve. If latency could not be reduced, the throughput could still be improved by reading from four banks at a time. Reading in parallel gave an amortized RAM latency of $200/4 = 50$ ns, which was fast enough.

Keep in mind that this architecture reduced the penalty of read operations, but plotting a pixel in the framebuffer with a write operation was still slow. Writing to the VRAM as little as possible was crucial to maintaining a decent framerate.

²⁶Digital to Analog Converter.

²⁷Computer Graphic: Principles and Practice 2nd Edition, page 168.

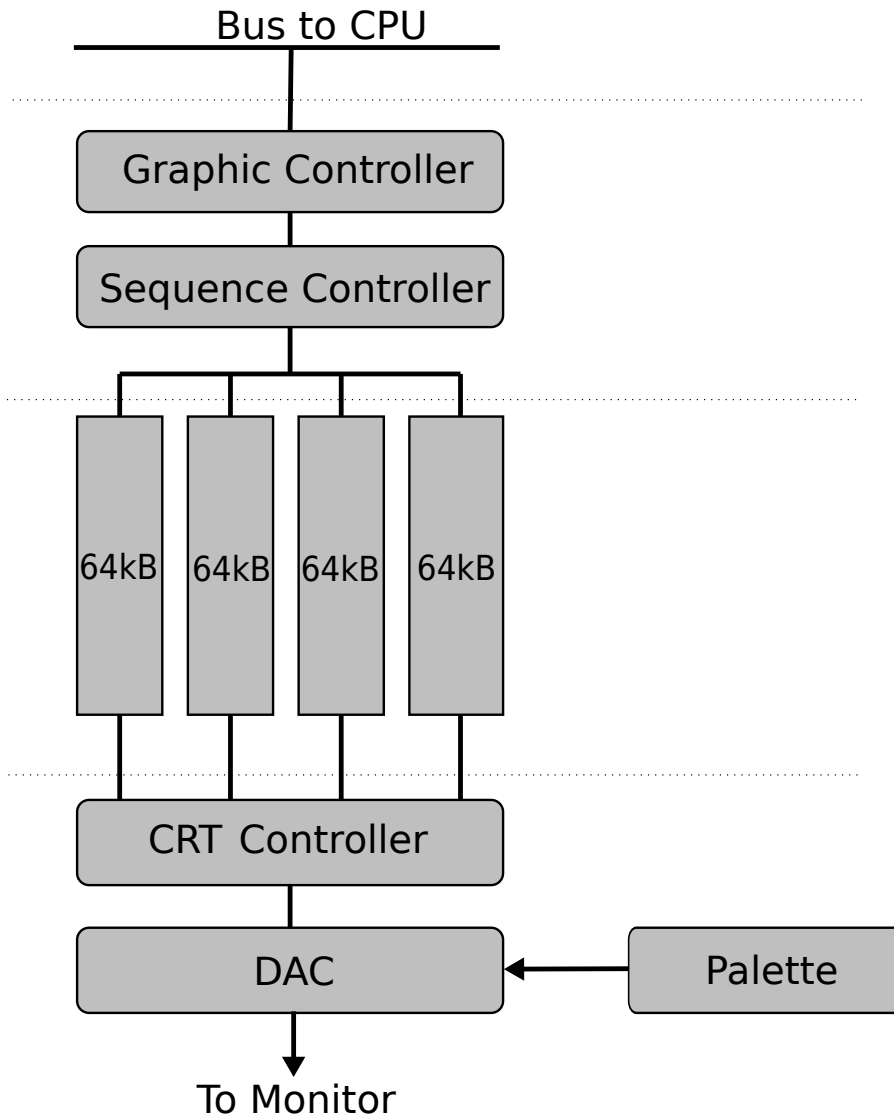


Figure 2.20: VGA Architecture.

2.3.3 VGA Planar Madness

Four memory banks grant enough throughput to reach high resolutions at 60Hz/70Hz. The price is complexity of programming, as acknowledged by even the best programmers of the time.

“

Right off the bat, I'd like to make one thing perfectly clear: The VGA is hard-sometimes very hard-to program for good performance.

Michael Abrash - Graphic Programming Black Book

”

The first problem with this design is that it is unintuitive. There is no linear framebuffer and figuring out which byte corresponds to which pixel on screen is difficult.

This type of architecture is called "planar". In mode 13h, where one pixel is encoded on one byte, to write four pixels next to each other on a line on the screen requires writing one byte in each bank. Each of these banks is mapped to the same UMA memory address. This layout is better explained with a drawing.

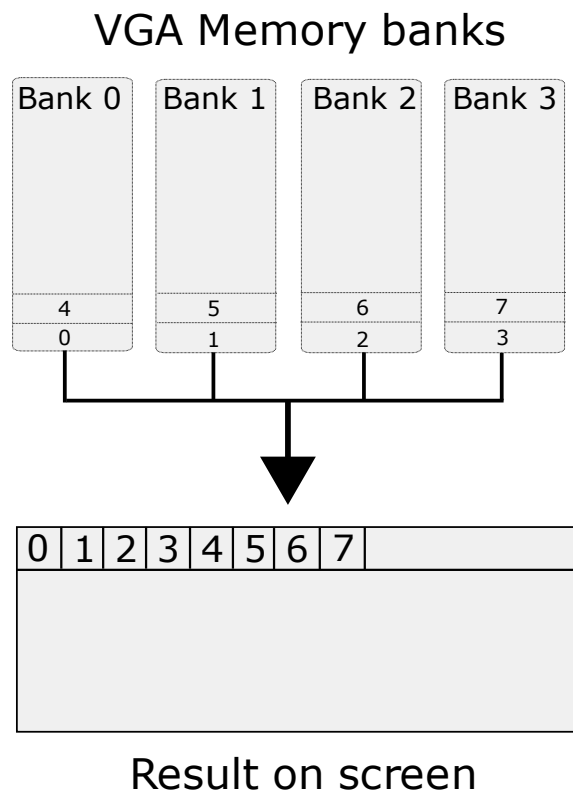


Figure 2.21: VGA mode 13h, How bank layout appears on screen.

In order to configure this mess of planes and the controllers, 50 poorly documented internal registers must be set. Needless to say few programmers dove into the internals of the VGA.

Figure 2.20, which described the architecture, was actually deceptively simplified. Figure 2.22 shows how IBM's reference documentation explained the VGA. The maze of wire showcases well the actual complexity of the system.

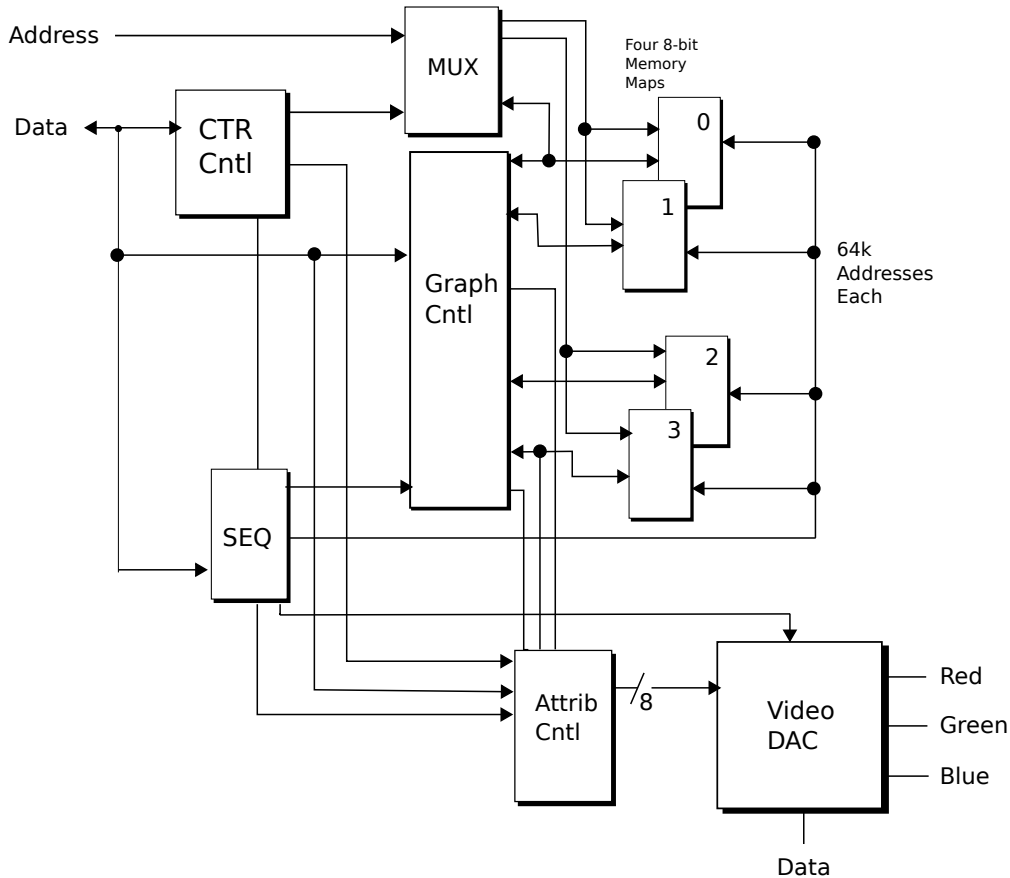


Figure 2.22: IBM's VGA Documentation.

To compensate for the complexity, IBM provided a routine to initialize all the registers via one BIOS call. One mode can be selected out of 15 available with an associated resolution, number of colors, and memory layout.

2.3.4 VGA Modes

The BIOS can be called to configure the VGA as follows.

Mode	Type	Format	Colors	RAM Mapping	Hz
0	text	40x25	16 (monochrome)	B8000h	70
1	text	40x25	16	B8000h	70
2	text	80x25	16 (monochrome)	B8000h	70
3	text	80x25	16	B8000h	70
4	CGA Graphics	320x200	4	B8000h	70
5	CGA Graphics	320x200	4 (monochrome)	B8000h	70
6	CGA Graphics	640x200	2	B8000h	70
7	MDA text	9x14	3 (monochrome)	B0000h	70
0Dh	EGA graphic	320x200	16	A0000h	70
0Eh	EGA graphic	640x200	16	A0000h	70
0Fh	EGA graphic	640x350	3	A0000h	70
10h	EGA graphic	640x350	16	A0000h	70
11h	VGA graphic	640x480	2	A0000h	70
12h	VGA graphic	640x480	16	A0000h	60
13h	VGA graphic	320x200	256	A0000h	70

Figure 2.23: VGA Modes available.

Programmers referenced VGA modes by their ID. It was common to see tutorials about mode 12h or mode 13h, which were the two best-suited modes for game programming.

2.3.5 VGA Programming: Memory Mapping

To write to the VRAM, the RAM's 1MiB address space maps 64KiB starting as indicated in table 2.23. In mode 13h for example, the VRAM is mapped from 0xA0000 to 0xFFFF. One of the first questions to come to mind is "How can I access 256KiB of RAM with only 64KiB of address space?" The answer is "bank switching" as summarized in figure 2.24. Write and Read operations are routed based on a mask register indicating which bank should be read or written to.

2.3.6 VGA Programming: Mode 12h

The first mode commonly considered for game programming is mode 12h. It offers a resolution of 640x480 at 60hz²⁸ with 16 colors. Each pixel is encoded in 4 bits (a nibble) spread

²⁸Because of bandwidth requirements, this mode is the only one to work at 60Hz. All others run at 70Hz.

across the four banks.

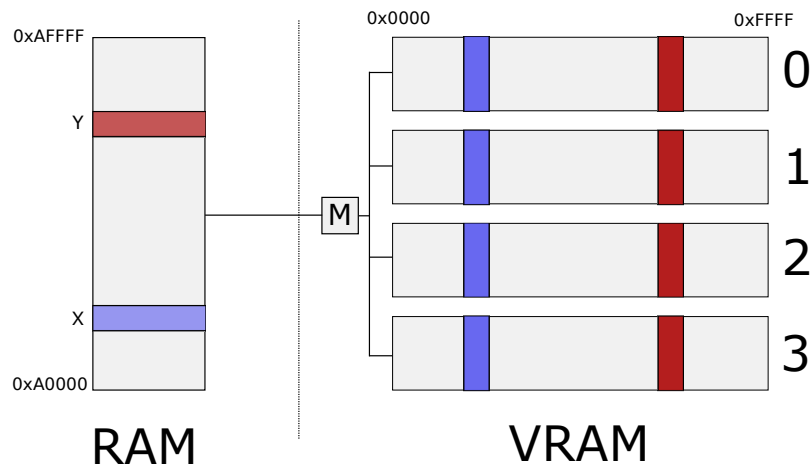


Figure 2.24: Mapping PC RAM to VGA VRAM banks.

To write the color of the first pixel, a developer has to write the first bit of the nibble in plane 0, the second in plane 1, the third in plane 2 and the fourth in plane 3. The CRT Controller then reads 4 bytes at a time (one from each plane) resulting in 8 pixels on screen.

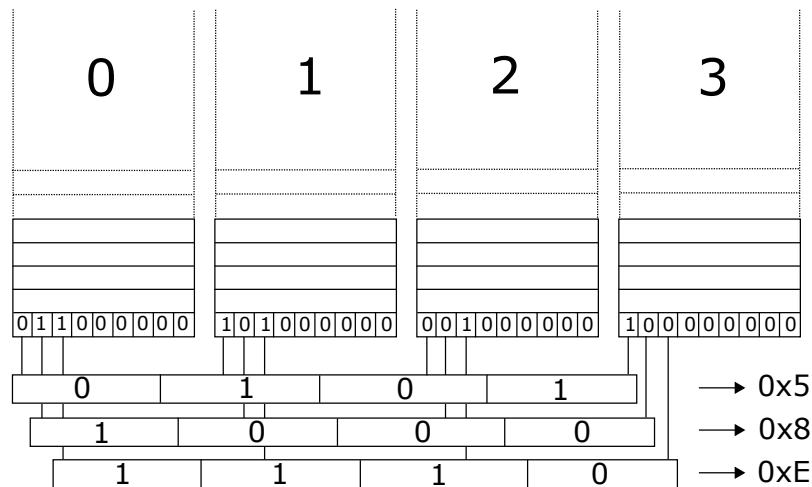


Figure 2.25: VGA bank layout in mode 12h.

The only good thing about this mode is that it has square pixels. The 640x480 aspect ratio matches the 4:3 of a CRT so there is no distortion of the framebuffer when it is output to the screen. Pretty much everything else is bad:

- No double buffer: $640 \times 480 / 2 = 0x25800$ bytes which is more than half the 256KiB (0x40000) of VRAM available.
- High resolution means more pixels, which means more calculations and more drawing.
- Sixteen colors severely limited artists. Games electing to use this mode usually looked worse than those using 256 color mode²⁹.



Figure 2.26: How Wolfenstein 3D would have looked if it used sixteen colors.

²⁹The exception to the rule is "Dark Seed", an adventure game developed and published by Cyberdreams in 1992. Inspired by H. R. Giger artworks, it is notorious for its gorgeous visuals in 640x480, 16 colors.

2.3.7 VGA Programming: Mode 13h

Mode 13h is far more appealing since it offers a lower resolution of 320x200 with 256 colors at 70Hz. It also has the advantage of faking linear buffer. A special chip called Chain-4 uses the lower 2 bits of the RAM address to automatically program the mask and route the operation to the appropriate VRAM bank. This convenience mechanism was originally added because mode 13h was meant to display static images and a linear address space made it easy for developers to copy from RAM to VRAM.

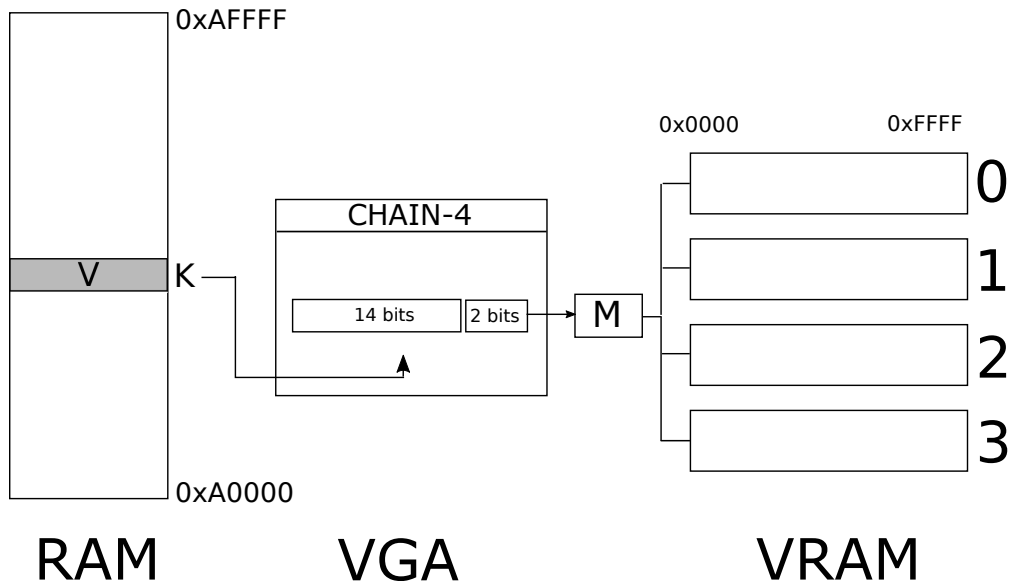


Figure 2.27: The Chain-4 chip routes I/O between RAM and VRAM.

When writing or reading a value V at address K in RAM, the Chain-4 breaks the address down into two parts:

- The 2 lower bits are used to configure the mask automatically. 00b goes to bank 0, 01b to bank 1, 10b to bank 2 and 11b to bank 3.
- The 14 higher bits are right shifted by two bits and used as an offset in the bank.

For example, writing at 0xABF13 in RAM would result in writing in bank 3 at offset 0xABF13 » 2 = 0x2AFC4. Since this is all hardwired in the Chain-4, this extra work is fast and totally invisible to the user.

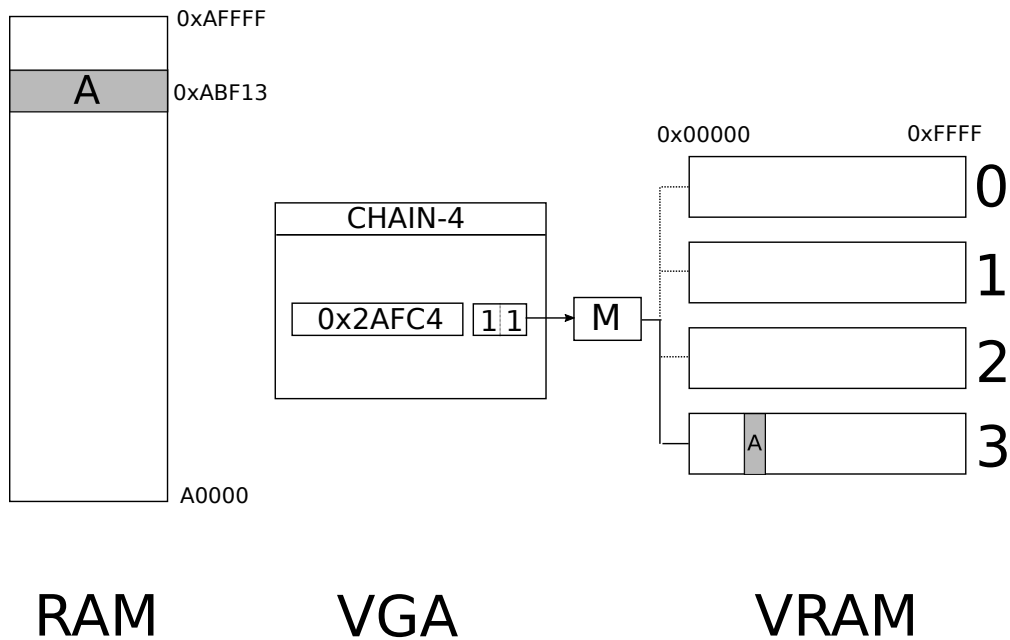


Figure 2.28

The side effect of this convenience mechanism is that 75% of the RAM is wasted (since only 14 bits are usable for offset).

2.3.7.1 Setup

To setup the VGA in Mode 13h using the BIOS is incredibly easy. It can be done with only two instructions:

```
mov ax,0x13 ; AH=0 (Change video mode), AL=13h (Mode)
int 0x10.   ; Video BIOS interrupt
```

The `int 10` instruction is a software interrupt caught by the BIOS routine in charge of graphic setup. It looks up the `ax` register to setup all 50 VGA registers with the corresponding mode.

After the VGA is initialized one can write to the mapped memory at 0xA0000. This can be demonstrated with a code sample; here is some code to clear the screen to black.

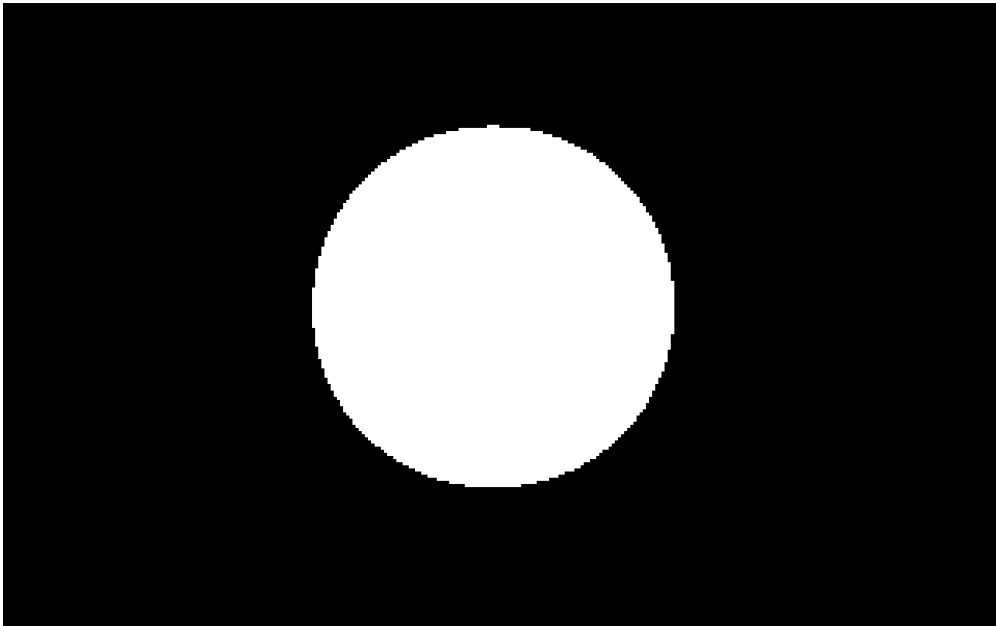
```
char far *VGA = (byte far*)0xA0000000L;

void ClearScreen(void){
    asm mov ax,0x13
    asm int 0x10

    for (int i=0 ; i < 320*200 ; i++)
        VGA[i] = 0x00;
}
```

Mode 13h looks a bit better than 12h but is still pretty terrible for games or even static images:

- With Chain-4 all the RAM address space is used and there is no way to double buffer.
- Since the resolution is 320x200, the aspect ratio (1.6) does not match the monitor's (1.333). As a result the framebuffer stored in the VRAM is stretched when transferred to the CRT. This may seem like a small distortion but is quite problematic. An example with a circle in the framebuffer displaying as an ellipse on the screen speaks for itself.



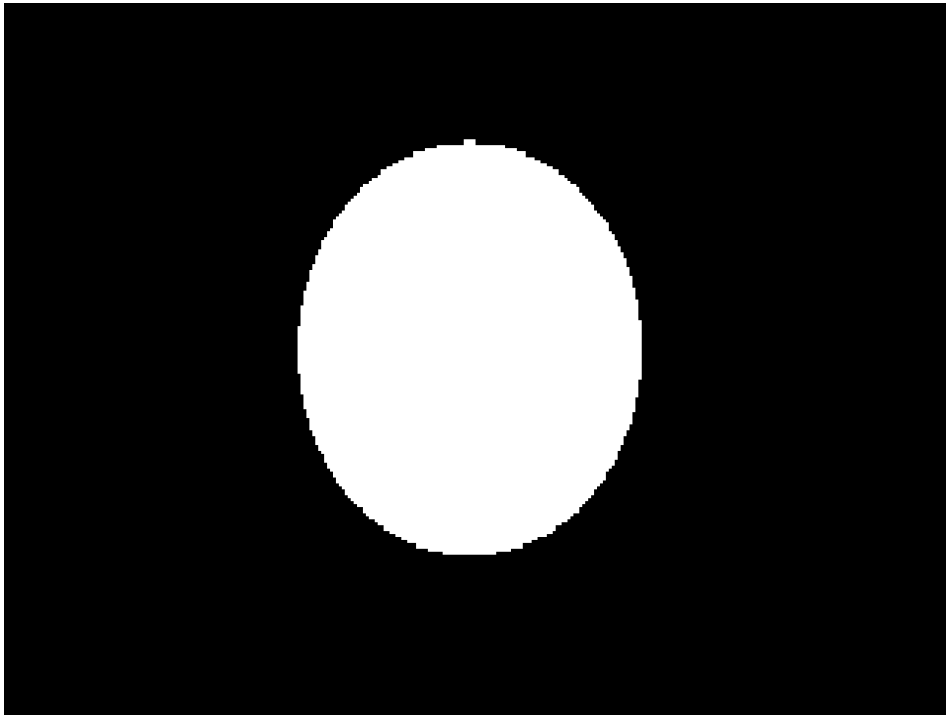
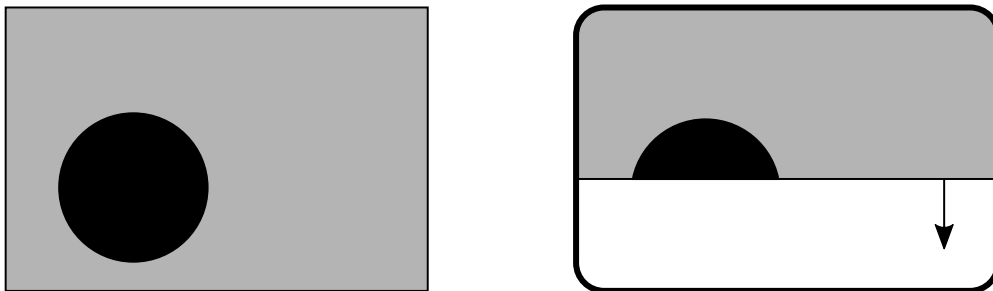


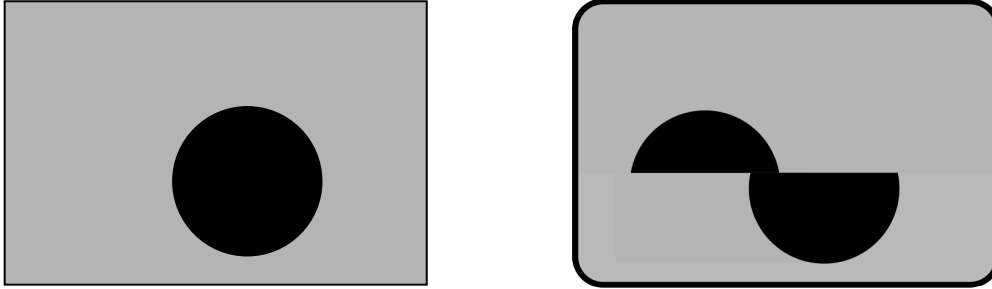
Figure 2.29: How the framebuffer appears once stretched on the CRT monitor.

2.3.8 The Importance of Double-Buffering

Double buffering has been mentioned often while describing the hardware, but so far we have not reviewed why it is paramount to achieving smooth animation. With only one buffer the software has to work at exactly the frequency of the CRT (70Hz). Otherwise a phenomenon known as "tearing" appears. Let's take the example of an animation rendering a circle moving from the left to the right:



In this example the CPU has finished writing the framebuffer (on the left) and the CRT's (on the right) electron beam has started to scan it onto the screen. At this point in time the electron beam has scanned half the framebuffer, so the circle has been partially drawn on the screen.

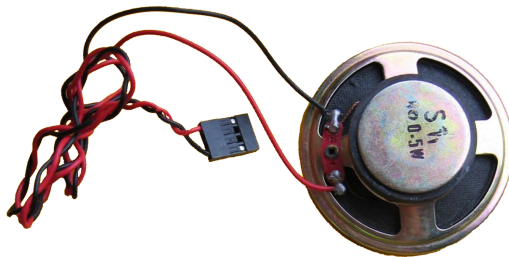


If the CPU is faster than the frequency of the CRT (70Hz), it can write the framebuffer again, before the scan is completed. This is what happened here. The next frame was drawn with the circle moved to the right. The electron beam did not know that and kept on scanning the framebuffer. The result on screen is now a composite of two frames. It looks like two frames were torn and taped back together. Hence the name "tearing".

With two buffers (a.k.a double buffering) the CPU can start writing in the second framebuffer without messing with the framebuffer being scanned to the screen³⁰. No more tearing!

2.4 Audio

PCs came equipped with a silver-dollar-sized beeper commonly known as a "PC Speaker", capable of generating a square wave via 2 levels of output.



³⁰Now the CPU speed is capped by the CRT refresh rate. Triple buffering can solve this at the price of frame latency.

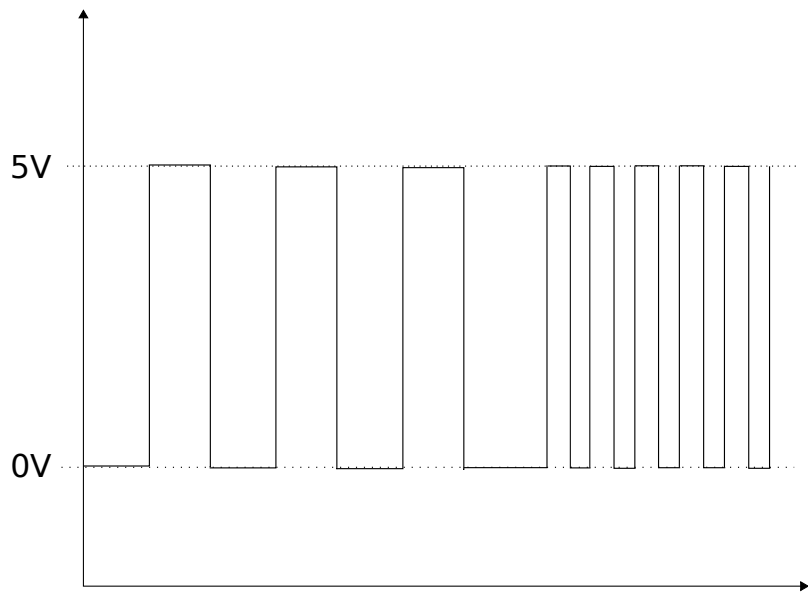


Figure 2.30: Two beeps of different frequencies generated via PC Speaker.

To this day, the PC speaker is the first output device to be activated during the boot process. The purpose of this primitive loudspeaker is to signal hardware problems with beep codes. It was intended to remain silent after a successful boot.

Beep Code	Meaning
No Beeps	Short, Bad CPU/MB, Loose Peripherals
One Beep	Everything is normal
Two Beeps	POST/CMOS Error
One Long Beep, One Short Beep	Motherboard Problem
One Long Beep, Two Short Beeps	Video Problem
One Long Beep, Three Short Beeps	Video Problem
Three Long Beeps	Keyboard Error
Repeated Long Beeps	Memory Error
Continuous Hi-Lo Beeps	CPU Overheating

However, square waves are not useful for producing anything pleasant. Some people saw a potential market and companies began manufacturing what were known as "sound cards". Users could buy these separately and insert them into one of the machine's ISA slots. These cards could be connected to real audio speakers via 3.5mm jacks and tremendously improved sound capabilities. In 1991, there were four cards on the market:

- AdLib music card
- SoundBlaster 1.0
- SoundBlaster Pro
- Disney Sound Source

Although adoption was growing (Creative would go on to sell one million SoundBlaster cards in 1991), the majority of PCs had no sound card which once again presented a huge problem for game developers.

2.4.1 AdLib

AdLib's music card was first on the market. The company was founded in 1988 by Martin Prevel, a former professor of music from Quebec. After an initial struggle to get game developers to use their card (the SDK was \$300), AdLib managed to convince Taito, Velocity, and Sierra On-Line to support their hardware. Sierra in particular did much to increase adoption with King's Quest IV selling close to 3 million copies. Soon after, all games supported the "music card".

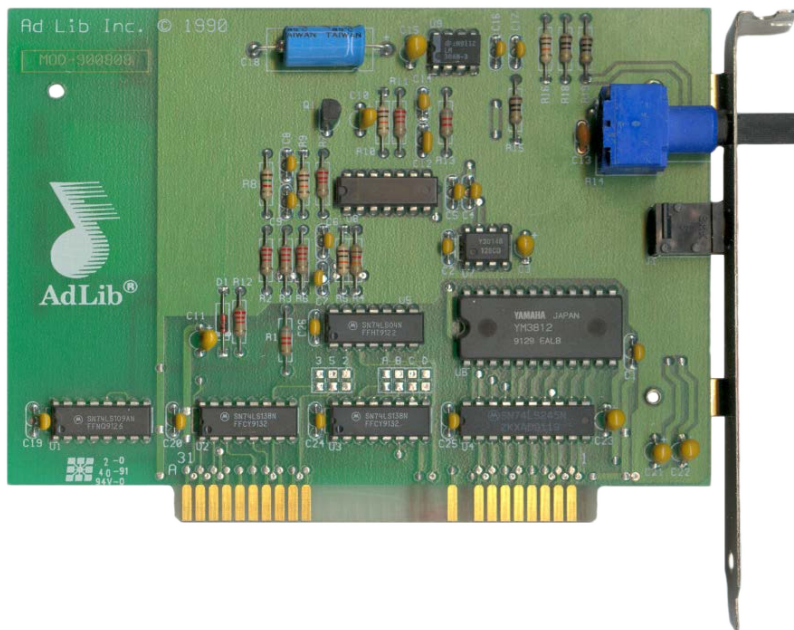


Figure 2.31: An AdLib sound card. Notice the big YM3812 chip and the 8-bit ISA connector.

Equipped with a Yamaha YM3812, also known as the OPL2, the card can produce 9 channels of sound, each capable of simulating an instrument. Based on FM synthesis, the channels were limited but allowed for pleasant music.

Trivia : Canadian companies, and especially those from Quebec, were prevalent in the early '90s due to their technological prowess. AdLib manufactured Sound Cards, Matrox made a killing with its Millenium Graphics Card, and Watcom sold the best DOS C compiler³¹. ATI³² would later emerge as a major GPU innovator in the 2000s.

2.4.2 Sound Blaster

The Sound Blaster 1.0 (code named "Killer Kard"), was released in 1989 by Creative. It was a smart product which was clearly targeting AdLib's dominant position.

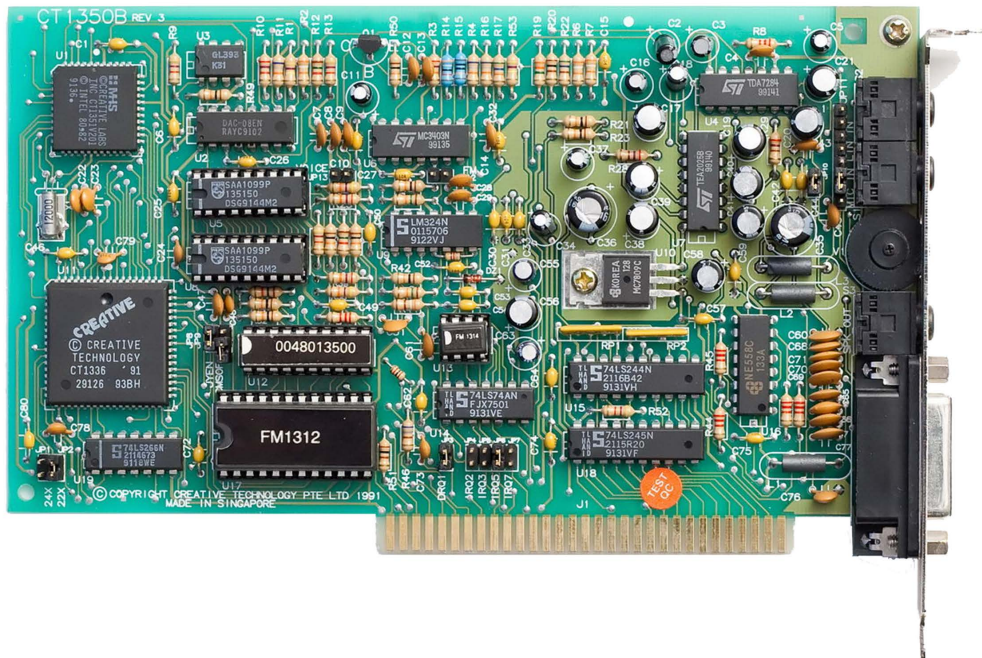


Figure 2.32: A SoundBlaster (v1.2).

³¹ Watcom's compiler was so good id would use it to compile Doom.

³²History would repeat itself in the late '90s in the field of graphic cards: Nvidia vs ATI.

Not only was it equipped with the same OPL2 chip, providing 100% compatibility with AdLib music playback, but it was also technologically superior with a DSP³³ allowing PCM playback (digitized sounds) at 8 bits per sample and up to 22.05kHz sampling rate. The card also came with a DA-15 port allowing joystick connection. Most importantly, the Sound Blaster was \$90 cheaper than the AdLib.

Figure 2.32 is the Sound Blaster model CT1350B. Notice the OPL2 chip (labeled FM1312), the big CT1336 bus interface (labeled "CREATIVE") on the center left, the CT1351 DSP on the upper left, and the 8-bit ISA bus connector.

Trivia : The numerous advantages of the Sound Blaster card over the AdLib made it the de-facto standard shortly after its release and eventually brought AdLib to bankruptcy³⁴.

2.4.3 Sound Blaster Pro

The Sound Blaster Pro had all the capabilities of a Sound Blaster 1.0 but added support for stereo 22.05 kHz playback and 44.1 kHz in mono. It also added a "mixer" to blend audio sources (mic, line in, and CD) and choose the attenuation level of left and right outputs. Stereo was achieved with a pair of YM3812 chips (one for each audio channel).

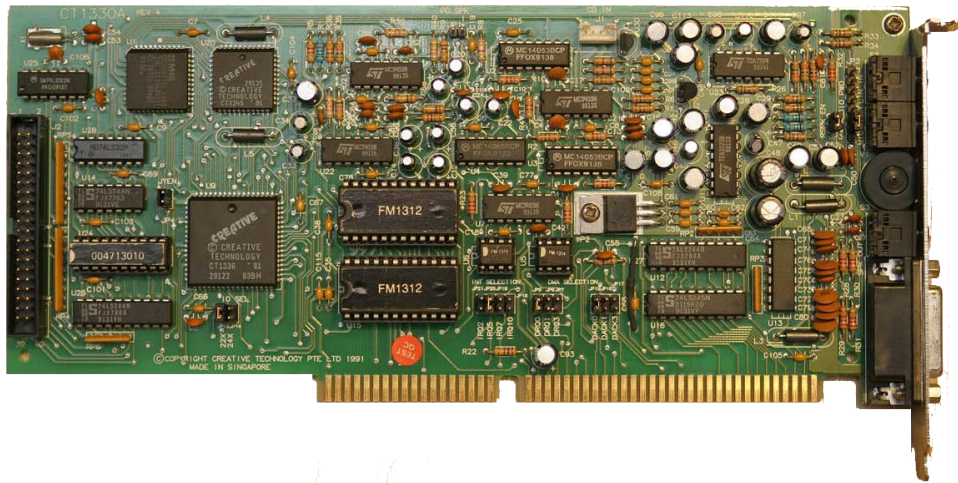


Figure 2.33: A Sound Blaster Pro card.

³³ An Intel MCS-51 "Digital Sound Processor", not "Digital Signal Processor".

³⁴ The reign of the Sound Blaster came to an end with Windows 95, which standardized the programming interface at application level and eliminated the importance of compatibility with Sound Blaster

Figure 2.33 shows a Sound Blaster Pro, model CT1330A. Most components were duplicated for stereo purposes. Notice the double FM1312 chip in the center. The two big chips in the upper left are the DSP (CT1341) and the Mixer. Below, labeled "Creative" is the big CT1316 bus interface.

Trivia : The card appears to have a 16-bit ISA bus connector (check the difference with the Sound Blaster 1.0). However it does not have 'fingers' for data transfer on the higher "AT" portion of the bus connector. It uses the 16-bit extension to the ISA bus to provide the user with an additional choice for an IRQ (10) and DMA (0) channel only found on the 16-bit portion of the edge connector.

Notice on the very left of the card in black there is a Panasonic interface to connect a CD-ROM. That was the only way to connect a CD-ROM to a PC back then.

2.4.4 Disney Sound Source

In 1990, Disney began selling the Disney Sound Source (DSS). Plugged into the printer port (parallel port) of the PC, an 8-bit DAC similar to the "Covox Speech Thing" was connected to a speaker box.



Figure 2.34: The speaker box (DAC not shown).

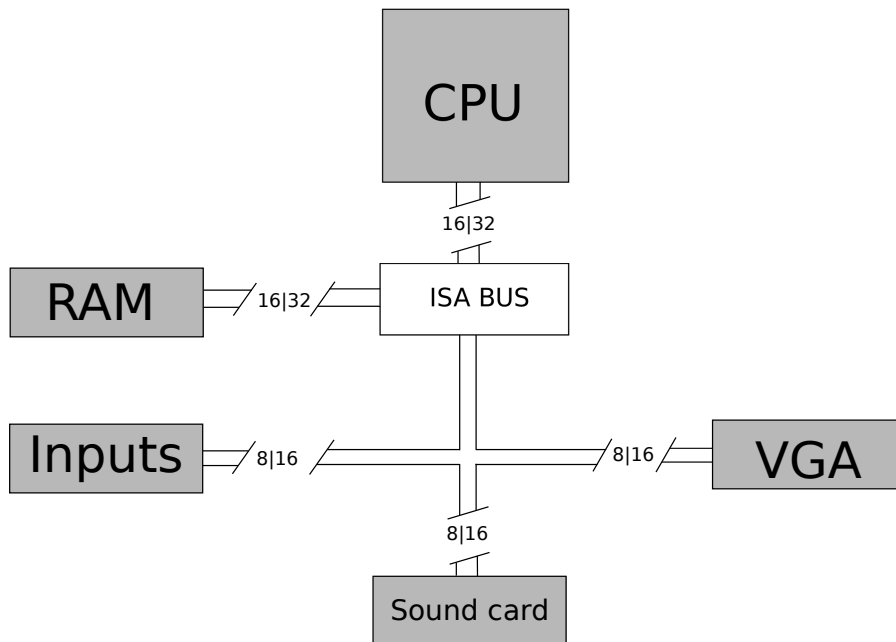
It was incredibly easy to set up, simple to program (it could only play one type of PCM and

had no FM synthesizer), and very cheap compared to the other audio solutions (\$14). It would have made programmers and customers happy if not for one serious limitation. The parallel port bandwidth³⁵ allowed a sampling rate up to 18,750 Hz but the design of the DSS limited the PCM sampling rate to 7,000Hz. This was still enough to produce pleasant sounds, but fell short when compared to the 22kHz or even 44kHz of a Sound Blaster Pro.

2.5 Bus

Although developers had no control over them, it is still worth mentioning how these components were connected to each other.

The ISA³⁶ bus connects the CPU to all devices, including RAM. It was already 10 years old in 1991 but still used universally in PCs. The data path to the RAM is either 16 bits wide for 286 and 386SX or 32 bits on 386DX based machines. It runs at the same frequency as the CPU.



³⁵The parallel port maximum bandwidth was 150 kbits/s at the time. Enhanced Parallel Port and later Enhanced Capability Port significantly increased the transfer rate necessary for scanners and laser printers.

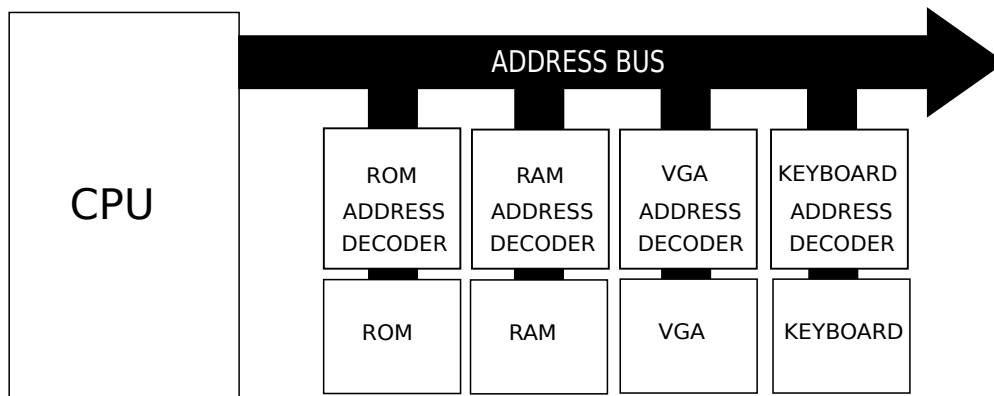
³⁶Industry Standard Architecture.

The rest of the bus connecting to everything that is not the RAM can be either:

- 8 bits wide at 4.77 MHz for 19.1 Mbit/s
- 16 bits wide at 8.33MHz for 66.7 Mbit/s³⁷.

It is also backward compatible and an 8-bit ISA card can be plugged into a 16-bit ISA bus.

Trivia : On ISA all devices are connected to the bus at all times and listen on the bus address lane. Each device features an "address decoder" to detect if it should reply to a bus request. This is how the VGA RAM is "mapped" in RAM. The VGA card "address decoder" filters out everything that is not within A0000h and AFFFFh. Accordingly, the RAM disregards any request that is within the range [A0000h - AFFFFh].



In practice the effective bandwidth of the bus is divided by two due to packet overhead and interrupts. As a result, a PC equipped with an 8 bit ISA VGA card can push $19.1\text{Mbit/s}/2/8 = 1.1\text{MB/s}$. In mode 13h, since a frame is $320 \times 200 = 64,000$ bytes, the theoretical maximum framerate with a CPU taking 0ms to render a frame is $1,100,000 / 64,000 = 17$ frames per second.

On a 16-bit VGA card, $33,400,000$ bits per seconds gives $33,400,000/8/64000 = 65$ frames per second.

If you factor in other things which had to be transported by the bus such as palette, keyboard interrupt, mouse inputs, and music/sounds (at 23kHz sampling a digitized sound effect consumes $23,000/70 = 328$ bytes per frame) it is easy to understand how important

³⁷https://en.wikipedia.org/wiki/List_of_device_bit_rates .

it was to limit data transfer and why few programs of that era could max out the VGA's 70 frames per second³⁸.

2.6 Inputs

At a time before the ubiquitous USB, inputs were a mess with no less than four ports, all programmed differently.

The parallel port (DB-25) was on every computer and usually used to connect dot-matrix printers (loud things that printed with needles). The parallel port was multi-purpose and the Disney Sound Source could be plugged into it.

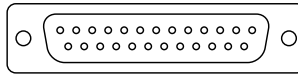


Figure 2.35: Parallel Port

The serial port (DE9) was used to connect the mouse.

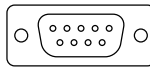


Figure 2.36: Serial Port

The PS/2 port was used to connect a keyboard.



Figure 2.37: PS/2 Port

Finally, a SoundBlaster sound card connected via the ISA bus provided a Game Port (DA-15) allowing for connection to a joystick³⁹.

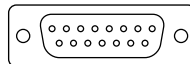


Figure 2.38: Game Port

³⁸Specialized demos could reach 70 fps on VGA by careful management of unchanged regions or using planar writes for the 4x speedup.

³⁹In 1981, the very first IBM PC could be purchased with a DA-15 "Game Port" extension card at the cost of \$55 (\$159 in 2018).

Trivia : The CRT monitor was connected to the VGA card via a DE15 port. More than 20 years later manufacturers are still trying to get rid of the "VGA port" as it is now commonly called. It looked a lot like a serial port and newcomers often damaged it by forcing a mouse into it.

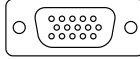


Figure 2.39: VGA Port

2.7 Summary

To say a PC was difficult to program for games would be an understatement. It was a nightmare. The CPU was good at doing the wrong thing, the best graphic interface allowed neither double buffering nor square pixels, the memory model only allowed 1 standard MiB with an address composed of two separate 16-bit registers, and the `near/far` pointers forbade using standard C. Last, but not least, the default sound system could only produce square waves and the people who did have a sound card installed could have any of the three major brands.

Yet despite all these unfavorable conditions, teams of developers gathered to tame the beast and unleash its power to gamers. One of these called themselves id Software⁴⁰.

⁴⁰They originally called themselves Ideas From the Deep but then decided to shorten it to simply id, which stands for "in demand", and is pronounced as in "did" or "kid." The name also refers to id, the part of the brain that behaves by the pleasure principle in Freudian psychology.

Chapter 3

Team

In 1990 a small company based in Shreveport, Louisiana was doing well in the shareware market. As a general software subscription service, Softdisk produced and mailed innovative titles to its members. The gaming division of Softdisk named "Gamer's Edge" produced titles for Apple II, Apple IIgs, C64, Mac, and PC every two months. Business was good but some of its employees had other ambitions. They thought they had the skills to make it big and they wanted to prove it.

They had created a new way to program side-scrolling. They called the technology "adaptive tile refresh" and it enabled hardware scrolling on a PC capable of rivaling a NES. In early 1990 they worked non-stop over a weekend to re-implement Super Mario 3 on a PC and demonstrate their skills to Nintendo. The team was successful in building a clone of Mario, but unfortunately "Ideas from the Deep" as they called themselves failed to convince Nintendo to give them a contract.

As impressed as they were, the Japanese firm wanted the Mario series to remain exclusive to Nintendo consoles.

“ We sent this demo to Nintendo of America, they in turn sent it to Kyoto to the mothership office, and the execs there saw the demo and were really impressed. However, they didn't want their intellectual property on anything but their own hardware, so they told us Good Job and You Can't Do This¹.

John Romero - Programmer

”

¹Super Mario Bros. 3 Demo (1990) by John Romero: <https://vimeo.com/148909578>



Figure 3.1: Mario 3 on PC. Notice the "IFD" for "Ideas From the Deep".

This episode was enough to convince them they had not only the talent to feed their ambitions, but also the teamwork and work ethic to potentially go all the way. On February 1st 1991², four Softdisk employees took the leap of faith and founded their own company. They named it "id Software"³.

Name	Age	Occupation
John Carmack	21	Programmer
John Romero	23	Programmer
Adrian Carmack	22	Artist
Tom Hall	28	Creative Director

Figure 3.2: id Software founding members.

²"id was founded Feb 1st, 1991", Tom Hall's Twitter account on Feb 1, 2020.

³"id" also stands for "In Demand." It is pronounced as in the Freud's id, ego, and superego

They immediately used the technology developed for "Mario 3 PC" to release their own titles and build their own intellectual property. Wasting no time, the team shipped no less than three titles annually.

- Commander Keen Episode 1, 2, and 3: *Invasion of the Vorticons* (December 14th, 1990)
- Commander Keen Episode 4, and 5: *Good Bye Galaxy* (December 15th, 1991)
- Commander Keen standalone game: *Aliens Ate My Baby Sitter* (December 1991)

The games, published by *Apogee Software*, were instant successes and sold very well. They also kept on writing games for *Softdisk*, most of which featured adaptive tile refresh⁴:

- *Dangerous Dave in the Haunted Mansion* (February 1991)
- *Commander Keen in Keen Dreams* (April 1991)
- *Rescue Rover* (1991)
- *Rescue Rover 2* (1991)
- *Shadow Knights* (1991)
- *Hovortank One* (May, 1991).
- *Catacomb 3-D* (November, 1991)

During Spring of 1991 the next generation of id Software technology started to surface⁵. *Hovortank One* placed the player inside a tank. There was no texture mapping yet and the pace was quite slow. *Catacomb 3-D* marked the introduction of textures and took immersion one step further by placing the player in control of a battle mage in first person view.

“

After *Catacomb 3-D* shipped at the end of Nov 1991, we were finishing *Commander Keen Goodbye, Galaxy* and launched it December 15, 1991. Then we took 2 weeks of vacation while John Carmack got the NeXT Computer and programmed a VGA vector quantization compression algorithm. *Wolf3D* started when the team was together in January 1992.

John Romero

”

⁴The *Softdisk* contract started in Feb 1991 and ended Jan 31, 1992. They had to keep making games, so they off-loaded two of them to focus on *Wolfenstein 3D*. "*Tiles of the Dragon*" and "*Scubaventure*" were those last two games made by contractors.

⁵You can see screenshots in the Appendix section on page 294 and 295.

In November 1991, the team was free from any obligations to Softdisk. Their next game was going to feature the 3D technology they were building and would be called Wolfenstein 3D. Four more people were added to the team for a total of eight.

Name	Age	Occupation
Jay Wilbur	30	Business
Kevin Cloud ⁶	27	Computer Artist
Bobby Prince ⁷	37	Composer
Jason Blochowiak ⁸	21	Programmer



Figure 3.3: The team as shown in Spear of Destiny, in an Easter Egg created by John Romero. To view this screen, the player had to go in the Change View menu, size the screen down one notch, hold down I and D, and press Enter.

cm esse easter egg foi descoberto

⁶Jay and Kevin were recruited in April 1992 but were credited anyway.
⁷Bobby had worked with id Software before on Keen 4-6 as a contractor but was never a full-time employee.
⁸Jason wrote part of the page manager and is credited for introducing John Carmack to Unix development, which ultimately led to the purchase of a NeXTstation Color. He left a few months after the creation of id.

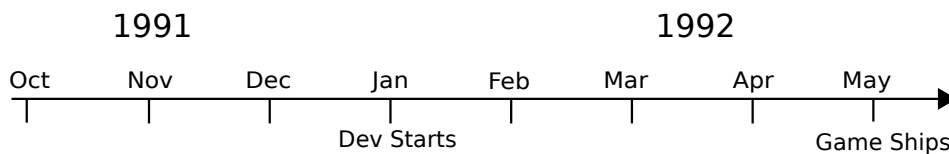


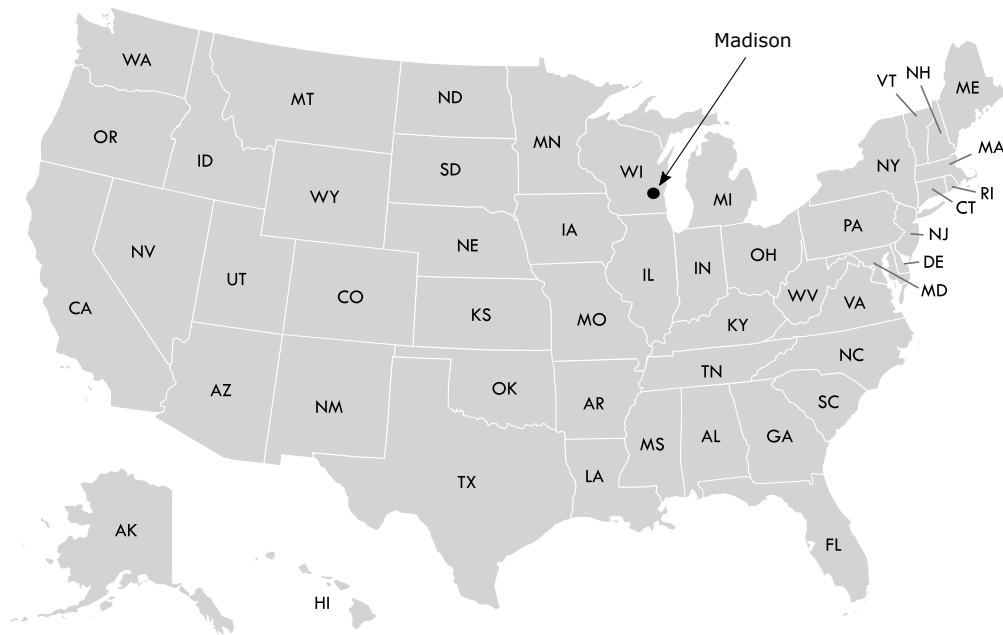
Figure 3.4: They were in fact wearing pants.

3.1 Organization

In September 1991, following Tom and Jason's high-school memories of the area, the team relocated from Shreveport, LA to Madison, WI. They established their office in a two story brick building at The Pines apartment complex, 2622 High Ridge Trail. They all lived in walking distance of the office except for John Carmack who, since he did not care, inhabited the second floor of the apartment.

The development of Wolfenstein 3D started in January 1992. As temperatures fell and snow dumped from the sky, the team kept itself increasingly busy and barely left the office. Development lasted four months and Wolfenstein 3D was released in May 1992.





During these four months, the organization of the team was pretty much standard for a game studio of this era: four guys crammed in one room, dictating a fast pace and strong sense of camaraderie (and a lot of noise disturbance given John Romero and Tom Hall's type of interaction).

On the map (next page) you can see on the upper floor the SNES where countless games of *F-Zero* were played and the Dungeons & Dragons area extensively mentioned in "*Masters of Doom*". To have a team member (John Carmack) with his apartment directly above the studio was not out of the ordinary⁹.

“

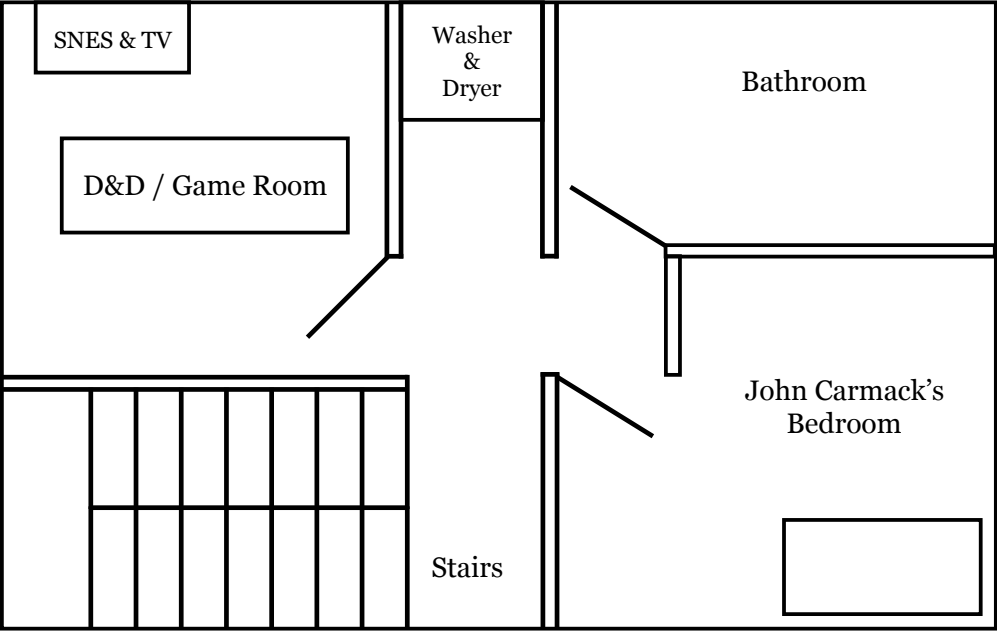
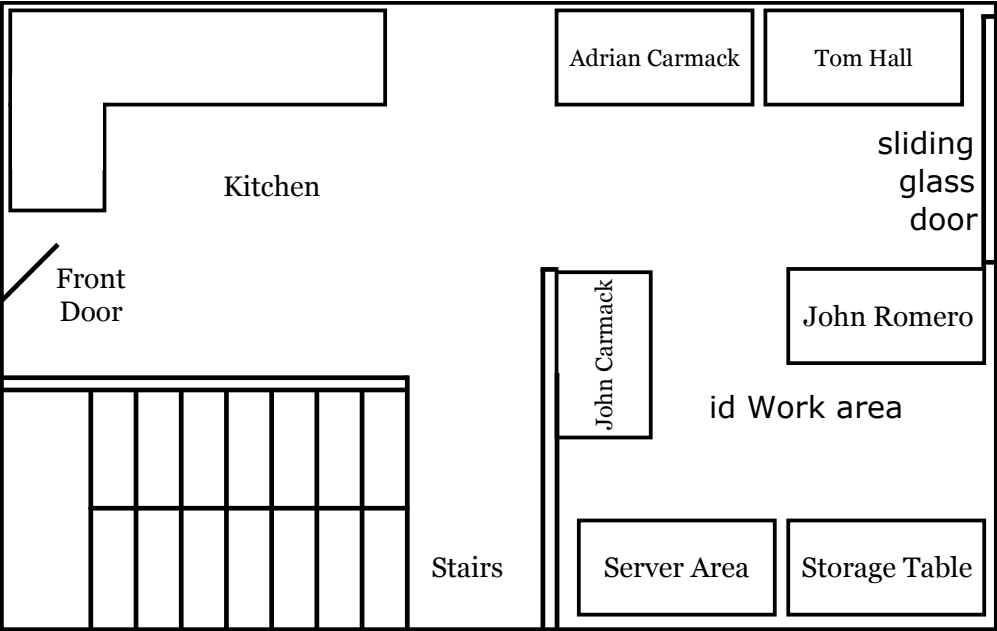
We started with floppy data transfer, but we had a Novell network on coax Ethernet by the end¹⁰. We didn't have a version control system. Surprisingly, we went all the way to Quake 3 without one, then we started using *Visual Source Safe*.

John Carmack - Programmer

”

⁹"90 Hours A Week And Loving It!" by Andy Hertzfeld.

¹⁰Comment from John Romero: "We bought a \$7000 Novell Netware 3.11 system while in Madison, WI in November 1991. This included the file server, cables, and network cards".

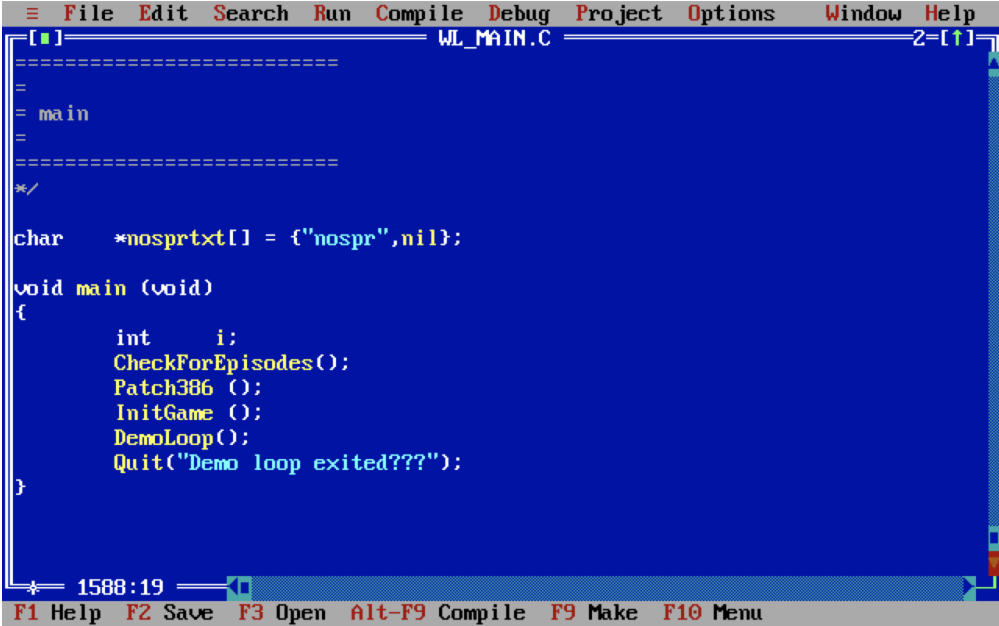


Everybody was working with the best PC money could buy, a high end 386-DX 33MHz with 4MiB of RAM.

3.2 Programming

Development was done with Borland C++ 3.1 (but the language used was C) which by default ran in VGA mode 3 offering a screen 80 characters wide and 25 characters tall.

John Carmack took care of the runtime code. John Romero programmed many of the tools (TED5 map editor, IGRAB asset packer, MUSE sound packer). Jason Blochowiak wrote important subsystems of the game (Input manager, Page manager, Sound manager, User manager).



```

≡ File Edit Search Run Compile Debug Project Options Window Help
[ ] WL_MAIN.C 2-[↑]
=====
=
= main
=
=====
%./
char  *nosprtxt[] = {"nospr",nil};
void main (void)
{
    int    i;
    CheckForEpisodes();
    Patch386 ();
    InitGame ();
    DemoLoop();
    Quit("Demo loop exited???");
}
* 1588:19
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

```

Figure 3.5: Borland C++ 3.1 editor

Borland's solution was an all-in-one package. The IDE, BC.EXE, despite some instabilities allowed crude multi-windows code editing with pleasant syntax highlights. The compiler and linker were also part of the package under BCC.EXE and TLINK.EXE¹¹.

¹¹Source: Borland C++ 3.1 User Guide.

There was no need to enter command-line mode however. The IDE allowed to create a project, build, run and debug.

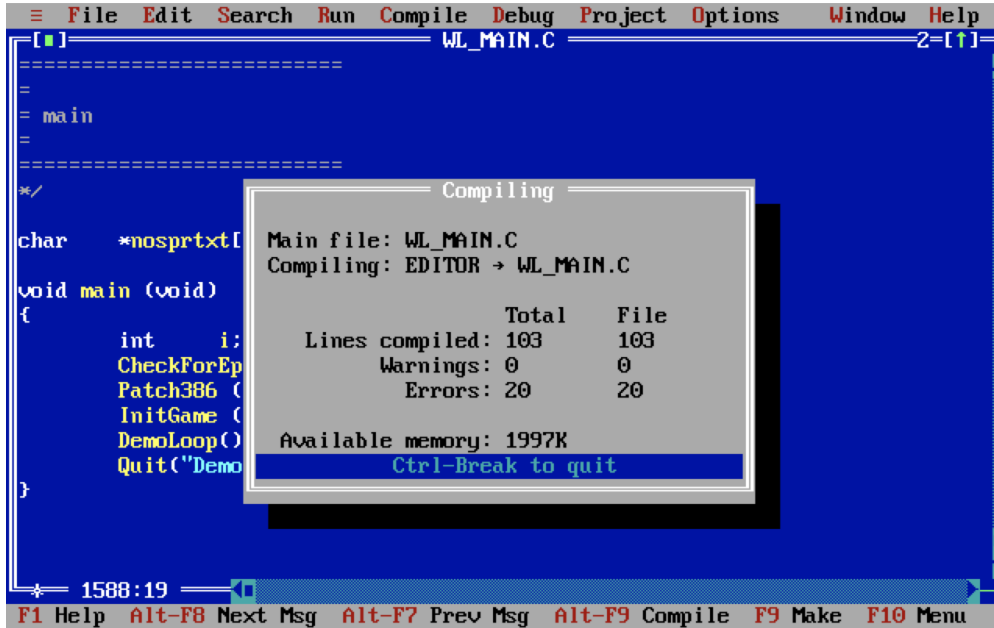


Figure 3.6: Compiling Wolfenstein 3D with Borland C++ 3.1

To compensate for the tiny CRT display, some of the developers used two screens¹².

“

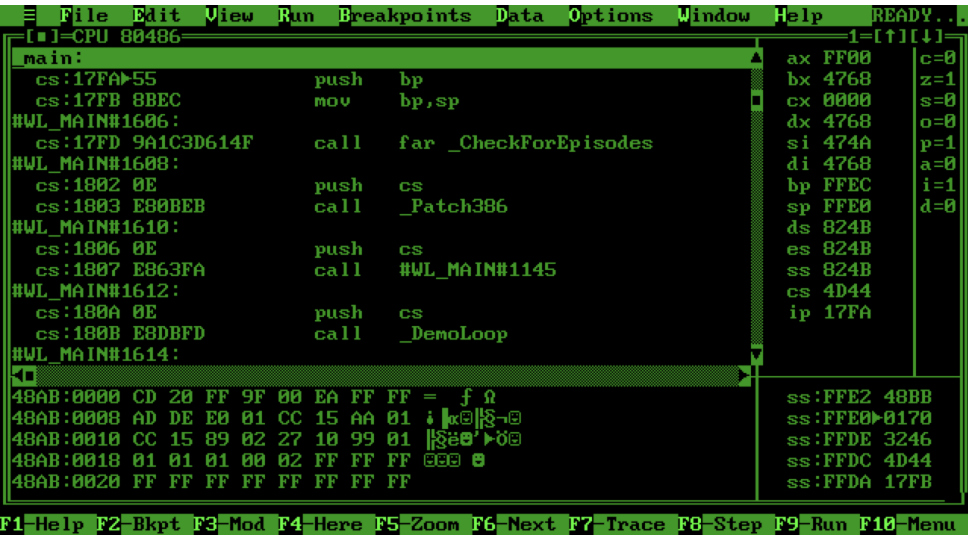
At that point, we wanted 21" monitors, but couldn't justify them. I used a second mono monitor to allow Turbo Debugger 386 to keep the main screen in graphics mode while I stepped through the code.

John Carmack - Programmer

”

You may have noticed in the listing of VGA modes that mode 13h and mode 3h (see Figure 2.23, p54) don't have the same starting RAM address. This allows for a trick where two graphics cards are plugged into the same PC. One MDA setup as monochrome text mode picks up data at 0xB8000 while a VGA mapped at 0xA0000 runs the game normally.

¹²Note from John Romero: "Both John and I had small 12" amber monochrome monitors we used with Turbo Debugger.

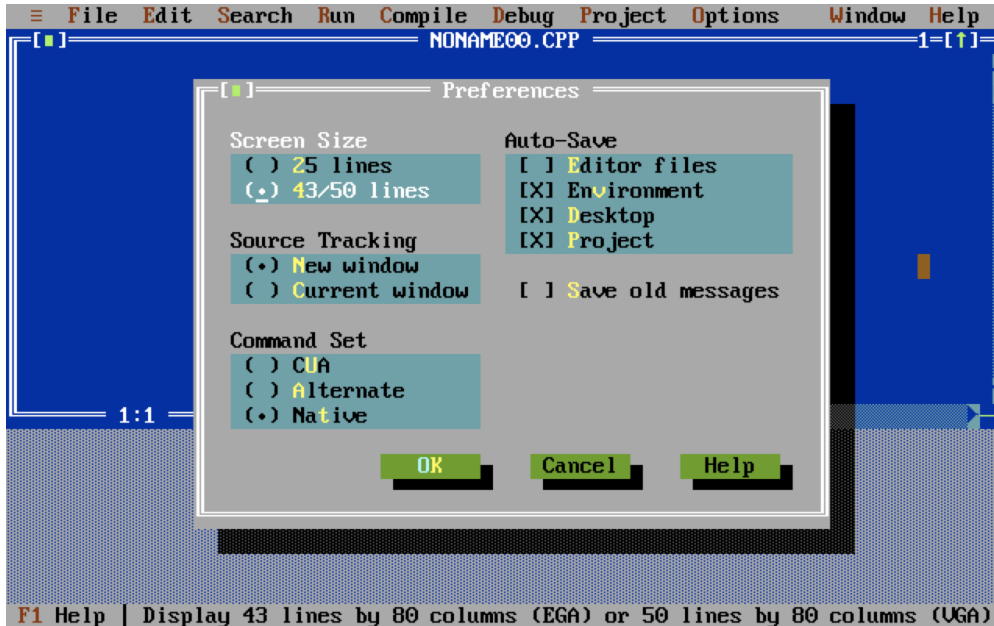


The screenshot shows the Borland 3.1 debugger interface. The main window displays assembly code for a program named 'main'. The code includes instructions like 'push bp', 'mov bp,sp', 'call far _CheckForEpisodes', 'push cs', 'call _Patch386', 'push cs', 'call #WL_MAIN#1145', 'push cs', and 'call _DemoLoop'. The right-hand pane shows the current state of registers: ax=FF00, bx=4768, cx=0000, dx=4768, si=474A, di=4768, bp=FFEC, sp=FFE0, ds=824B, es=824B, ss=824B, cs=4D44, and ip=17FA. Below the registers, memory addresses and their contents are listed, such as 48AB:0000 containing CD 20 FF 9F 00 EA FF FF.

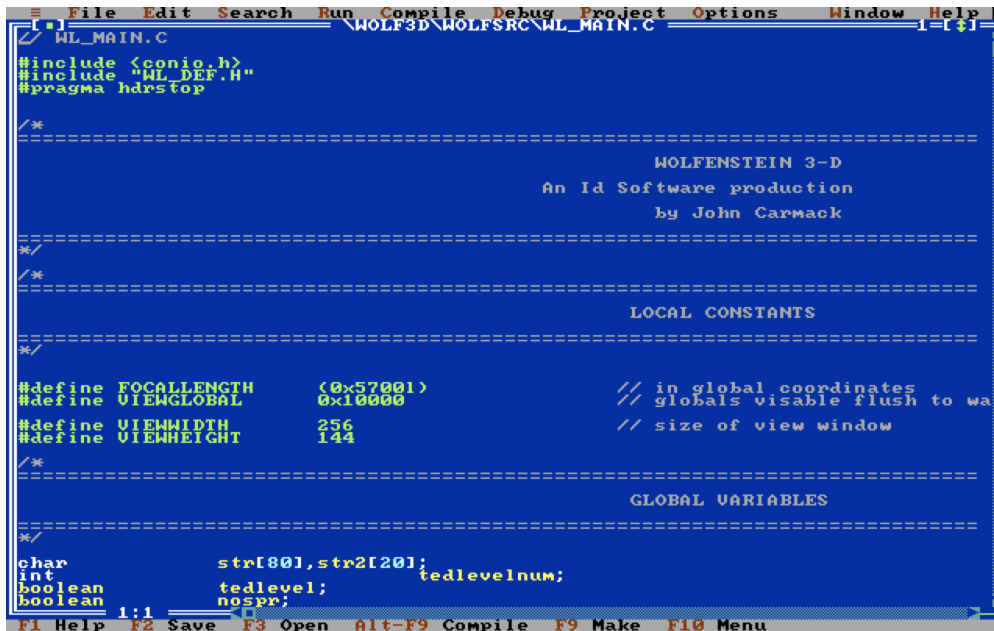
Above, MDA screen with Borland 3.1 debugger. Below, the VGA screen running the game.



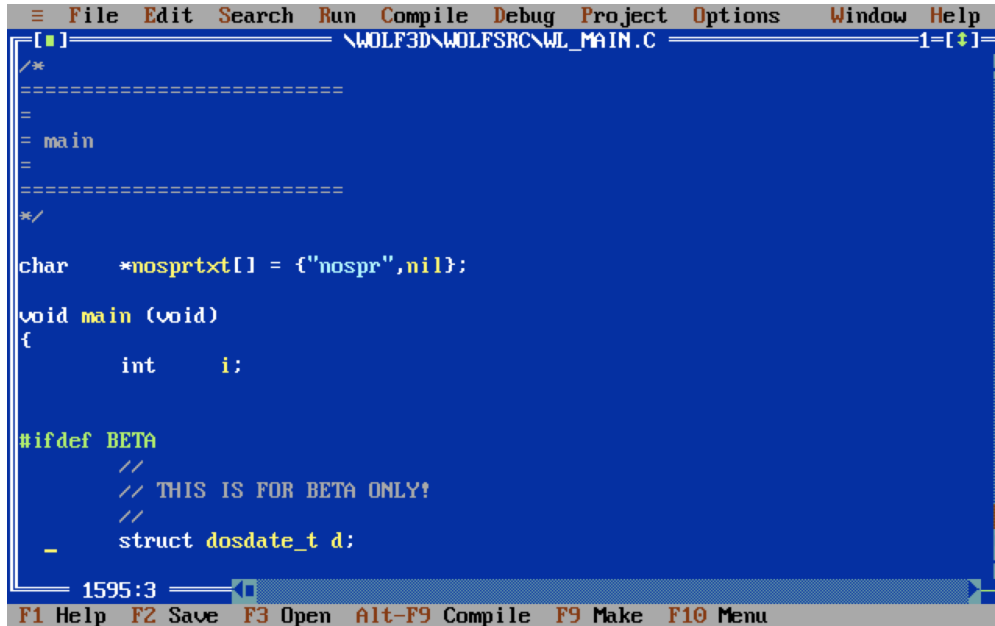
Another way to improve screen real estate was to use "high resolution" 50x80 text mode.



The comments still fit perfectly on screen since only the vertical resolution is doubled.



The file `WL_MAIN.C` opened in both modes demonstrates the readability/visibility trade-off.



```

File Edit Search Run Compile Debug Project Options Window Help
[.] \WOLF3D\WOLF3SRC\WL_MAIN.C 1=[+]
/*
=====
=
= main
=
=====
*/

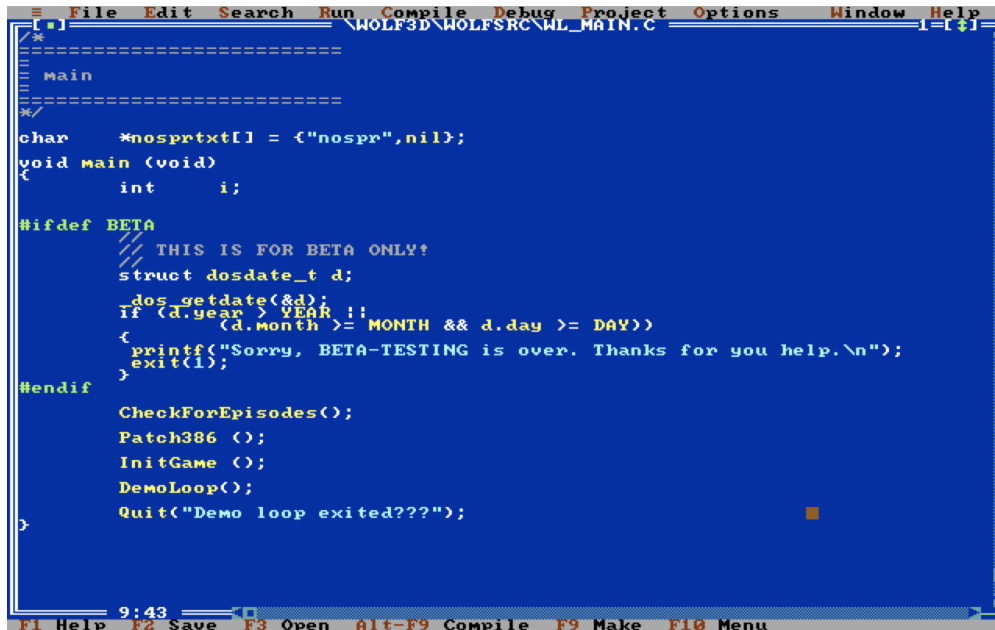
char *nosprtxt[] = {"nospr",nil};

void main (void)
{
    int i;

#ifdef BETA
    //
    // THIS IS FOR BETA ONLY!
    //
    struct dosdate_t d;

1595:3
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

```



```

File Edit Search Run Compile Debug Project Options Window Help
[.] \WOLF3D\WOLF3SRC\WL_MAIN.C 1=[+]
/*
=====
=
= main
=
=====
*/

char *nosprtxt[] = {"nospr",nil};
void main (void)
{
    int i;

#ifdef BETA
    //
    // THIS IS FOR BETA ONLY!
    //
    struct dosdate_t d;
    _dos_getdate(&d);
    if (d.year > YEAR ||
        (d.month >= MONTH && d.day >= DAY))
    {
        printf("Sorry, BETA-TESTING is over. Thanks for you help.\n");
        exit(1);
    }
#endif

    CheckForEpisodes();
    Patch386 ();
    InitGame ();
    DemoLoop();
    Quit("Demo loop exited???");
}

9:43
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

```

3.3 Graphic Assets

All graphic assets were produced by Adrian Carmack¹³. All of the work was done with **Deluxe Paint** (by Brent Iverson, Electronic Arts) and saved in **ILBM**¹⁴ files (Deluxe Paint proprietary format).



Figure 3.7: Deluxe Paint was used to draw all assets in the game.

Since the VGA is palette-based (colors were not specified via 24-bit RGB but via indices pointing to a 256-color table) the creative process was difficult. Adrian had to first make the key decision of which colors would go in the palette¹⁵, then draw everything with only those colors.

¹³Kevin Cloud did a few textures and also worked on the design and layout of the Wolfenstein 3D hint book.

¹⁴InterLeaved BitMap.

¹⁵Some games like "Monkey Island" used multiple palettes depending on the section of the game. id Software went for a simpler solution with one palette for the whole game.

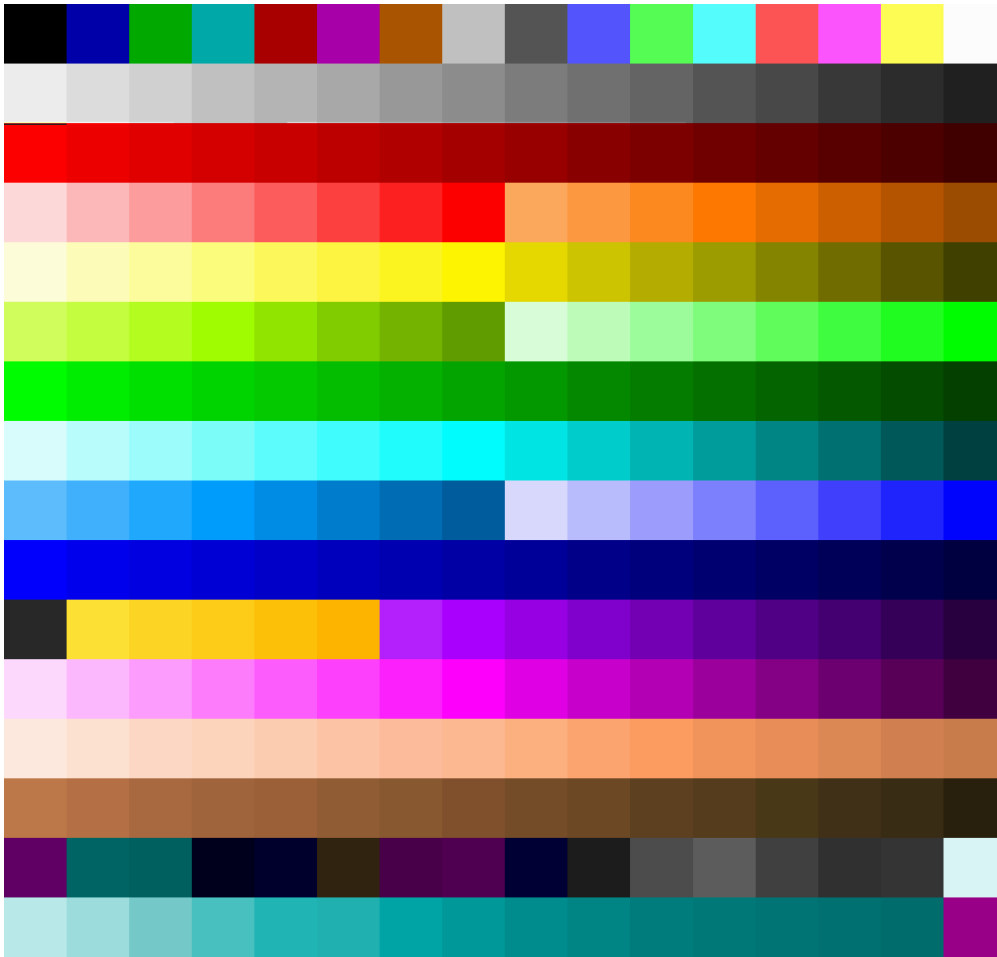


Figure 3.8: Wolfenstein 3D palette. Everything in the game is drawn using these 256 colors.

The palette coordinates run `0x00` to `0x0F` horizontally and `0x00` to `0xF0` vertically. The horizontal blue gradient at the bottom starts at `0xF0` and ends at `0xFE`. `0xFF` (represented in pink) is a special color deemed transparent by the engine and always skipped during rendering.

All assets were hand drawn with a mouse. Since the VGA stretched the framebuffer when displaying it on the screen, Adrian had to be careful to draw at the same resolution as the game would run (320x200).

“ Adrian and Kevin both worked directly in Deluxe Paint, we didn’t have any scanning tools at the time.

John Carmack - Programmer

”

Graphic assets are divided in two categories:

- 2D Menu items shipping with the game in VGAGRAPH, VGAHEAD, and VGADICT
- 3D Action phase items (walls and sprites) shipping in the VSWAP archive

3.4 Assets Workflow

After the graphic assets were generated, a tool (IGRAB) packed all ILBMs together in an archive and generated a C header file with asset IDs. The engine references an asset directly by using these IDs.

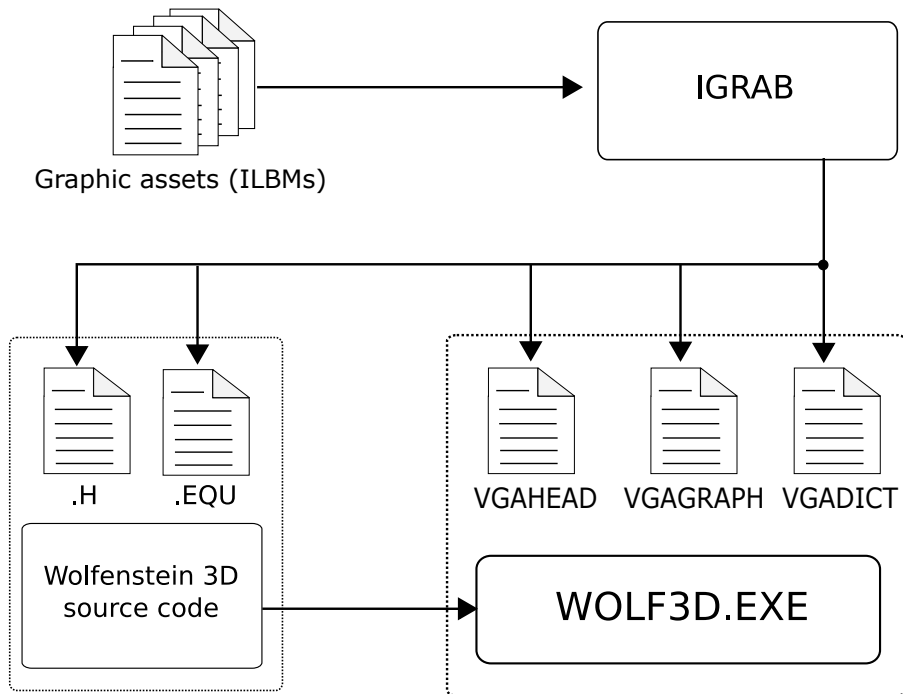


Figure 3.9: Asset creation pipeline for 2D Menu items

```

////////////////////////////////////
//
// Graphics .H file for .WL1
// IGRAB-ed on Sun May 03 01:19:32 1992
//
////////////////////////////////////

typedef enum {
    // Lump Start
    H_BJPIC=3,
    H_CASTLEPIC,                // 4
    H_KEYBOARDPIC,              // 5
    H_JOYPIC,                   // 6
    H_HEALPIC,                  // 7
    H_TREASUREPIC,              // 8
    H_GUNPIC,                   // 9
    H_KEYPIC,                   // 10
    H_BLAZEPIC,                 // 11
    H_WEAPON1234PIC,            // 12
    H_WOLFLOGOPIC,              // 13
    ...
    PAUSEDPIC,                  // 140
    GETPSYCHEDPIC,              // 141
}

```

In the engine code, asset usage is hardcoded via an enum. This enum is an offset into the HEAD table which gives an offset in the DATA archive. With this indirection layer, assets could be regenerated and reordered at will with no modification in the source code.

The DICT file contains the Huffman tree to decompress the assets, this is further discussed in the Cache Manager section on page 118.

```

void CheckKeys (void) {
    if (Paused) {
        LatchDrawPic (20-4,80-2*8,PAUSEDPIC);
        ...
    }
}

void PreloadGraphics(void) {
    LatchDrawPic (20-14,80-3*8,GETPSYCHEDPIC);
    ...
}

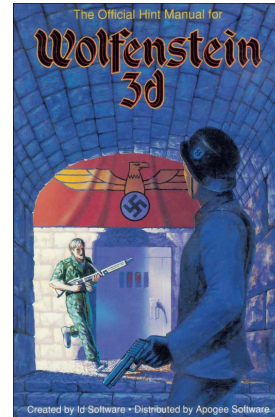
```

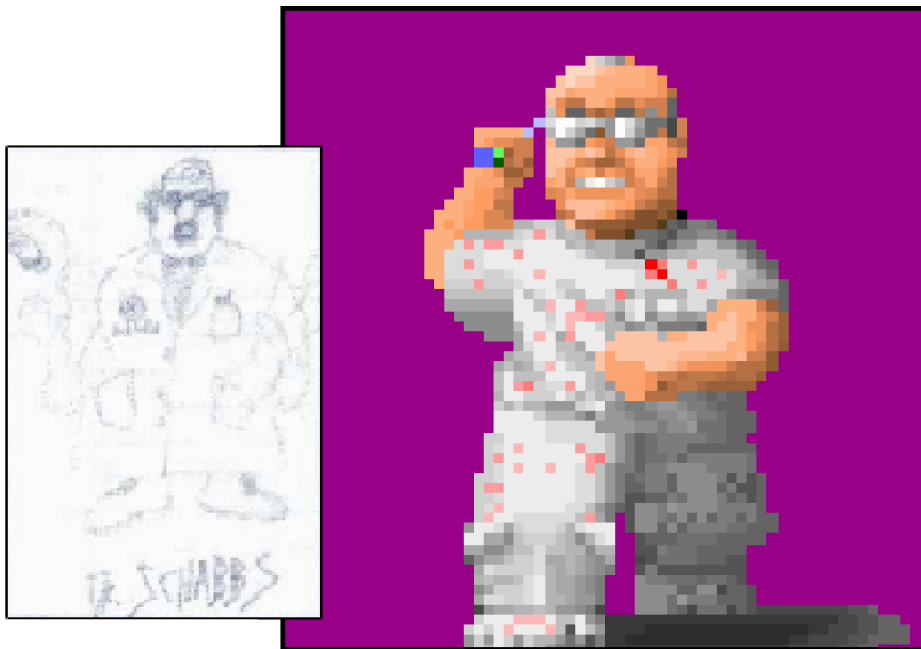
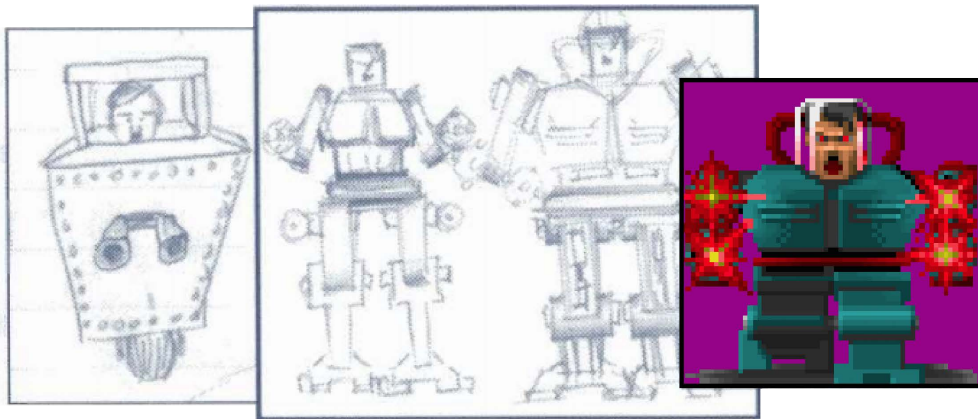
Trivia : This system led to issues when the source code was released. The .h header files provided did not match the asset files from the shareware or early versions of Wolfenstein 3D. The headers released were from *Spear of Destiny*. You can see the kind of graphic mess this led to in the article ["Let's compile like it's 1992"](http://fabiansanglard.net) on fabiansanglard.net.

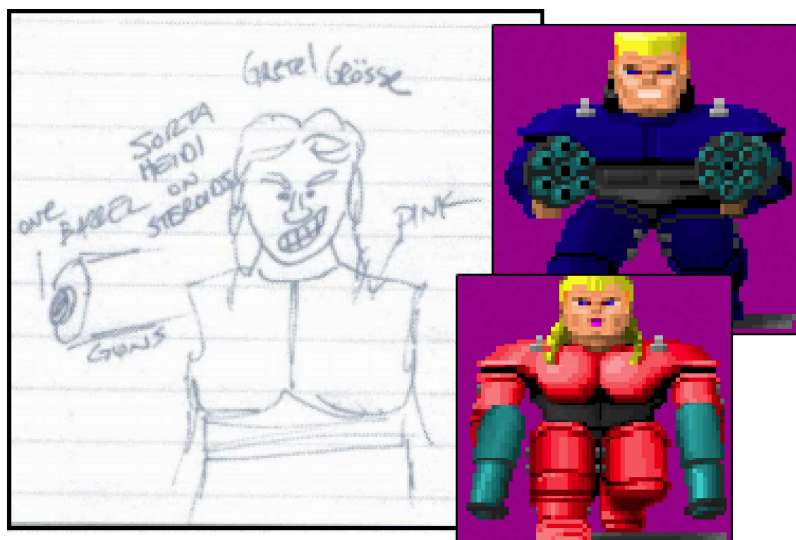
"The Official Hint Manual for Wolfenstein 3D" published in 1992 explains the creative process. It contains many drawings from Tom Hall and shows many behind the scene sketches made by Tom Hall and pixelarted by the graphic team.

“ When Id's Creative Director, Tom Hall gets an idea for a screen, he provides a sketch for Adrian Carmack. Below are some of Tom's early designs for the title screen. The third sketch was chosen.

”









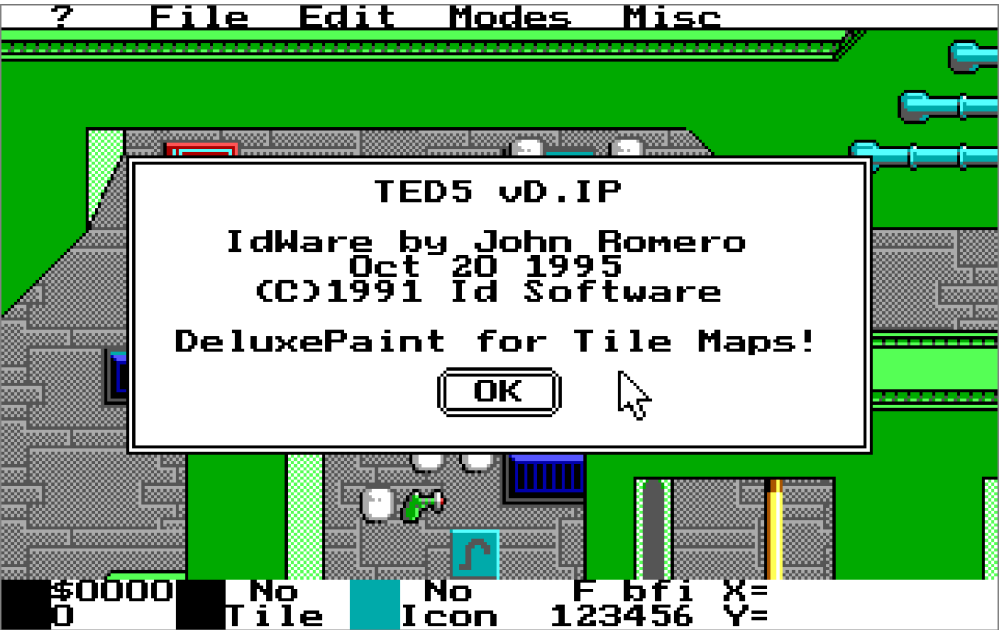
The hint manual also contains several photos of the team back in the day; it is worth a read for context¹⁶.

3.5 Maps

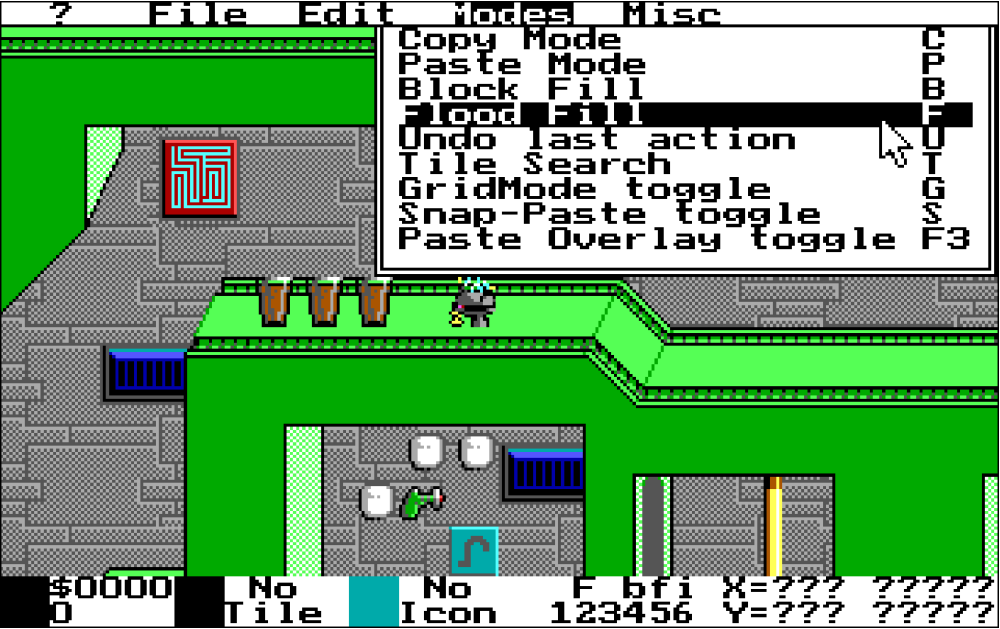
Maps were created using an in-house editor called TED5, short for Tile EDitor. TED5 was not created specially for Wolf 3D, but was originally made for the Commander Keen series and improved over the years. It was a versatile tool since it allowed for creating maps of both side-scroller games and top-down games like *Rescue Rover* and Wolf 3D.

TED5 is not stand-alone; in order to start, it needs an asset archive and the associated header (as described in the graphic asset workflow Figure 3.9 on page 85). This way, texture IDs are directly encoded in the map.

¹⁶The manual was designed on a *NeXT ColorStation*. It was the only usage of Steve Jobs's machine for Wolfenstein 3D even though id purchased it in December 1991. NeXT's series of workstations later became a key element of the production pipeline for Doom in 1993.



Trivia : The suffix, "vD.IP", was put in by the Rise of the Triad team in 1994. It stood for "Developers of Incredible Power".





TED5 allows placement of tiles on layers called "planes". This layered approach proved powerful and versatile. In Commander Keen, layers are used for background, tiles which the hero can stand on, generating bonuses and so on. For Wolfenstein 3D, two layers are used: one for walls, and one to place bonuses and enemies on.

Reusing TED5 was a double win. Not only did it save tool development time, but ramp-up time was also reduced as all team members had been using it for years. TED5 was so good at doing the job that it allowed designers to make a level within minutes¹⁷.

“ After talking with Romero and Tom, Scott learned that it was taking the group only about one day to make a level of the game. Ka-chung! Dollar signs! Instead of just three episodes, why not have six? Scott said, "If you can do thirty more levels, it would only take you fifteen days. And we could have it where people could buy the first trilogy for thirty-five dollars or get all six for fifty dollars, or if people buy the first episode and later want the second episodes it will be twenty dollars. So there's a reason to get them all!" After some consideration, id agreed.

- Masters of Doom

”

John Romero and Tom Hall did all the map design with TED5. Bobby Prince helped too and is credited for maps E6M2 and E6M3.

Trivia : The source code of TED5 was released several years later. Among the source code made of .C and .H was a mysterious _TOM.PIC¹⁸. It turned out to be an adult caricature of Tom Hall made by Adrian Carmack. The explanation came from John Romero:

“ "Hahahaha! Wow, I forgot all about that picture. I can't believe it's in the TED5 source files! It's basically a pic that Adrian drew of Tom [...] saying "Sorry!".

It's because Tom and Adrian used to share a worktable together. Tom would always bump the table while Adrian was drawing graphics with the mouse and Tom would say, "Sorry!".

John Romero - Programmer

”

¹⁷TED5 was used in 33 commercially shipped games (including Rise of the Triad in 1994).

¹⁸Intentionally not reproduced here.

3.6 Audio

3.6.1 Sounds

As mentioned in Chapter 2.4, audio hardware was highly fragmented. id decided to support four sound cards and the default PC speaker, which meant generating assets multiple times for each and packing them together with an in-house tool called MUSE into an AUDIOT archive (an id software proprietary format):



Figure 3.10: MUSE splash screen.

Three sets of each audio effect shipped with the game:

1. For PC Speaker
2. For AdLib
3. For SoundBlaster, SoundBlaster Pro, and Disney Sound Source

All voices were recorded by John Romero and Tom Hall faking German accents¹⁹ the best they could²⁰.

3.6.2 Music

All music composition was done by Bobby Prince.

“

In the early days of the OPL soundcards, the "gold standard" sequencing software was Sequencer Plus Gold ("SPG") by Voyetra. The reason for this was it had an OPL instrument/instrument bank editor.

To rough out compositions, I used Cakewalk ("CW"). I had been using it for several years already and had it all set up to use the analog boxes for sound output. Having "real" sounds from those boxes helped me visualize (audiolize?) what I wanted musically. I would save the CW files in *.mid format and load them into SPG to create the OPL instrument for each track. I built different instrument banks for the different genres of music.

Bobby Prince - Composer

”

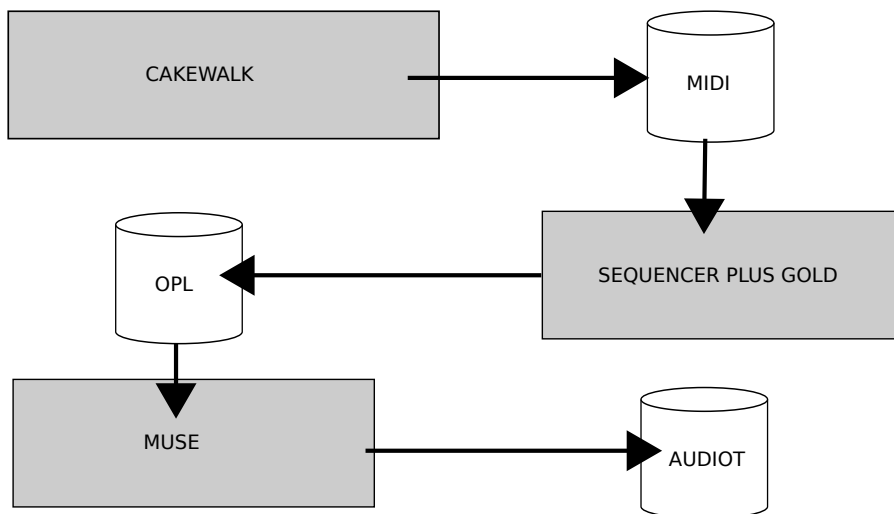


Figure 3.11: Music pipeline as described by Bobby Prince.

¹⁹Masters of Doom by David Kushner.

²⁰You can actually recognize their voices if you listen carefully: Tom Hall's "guten tag" is memorable.

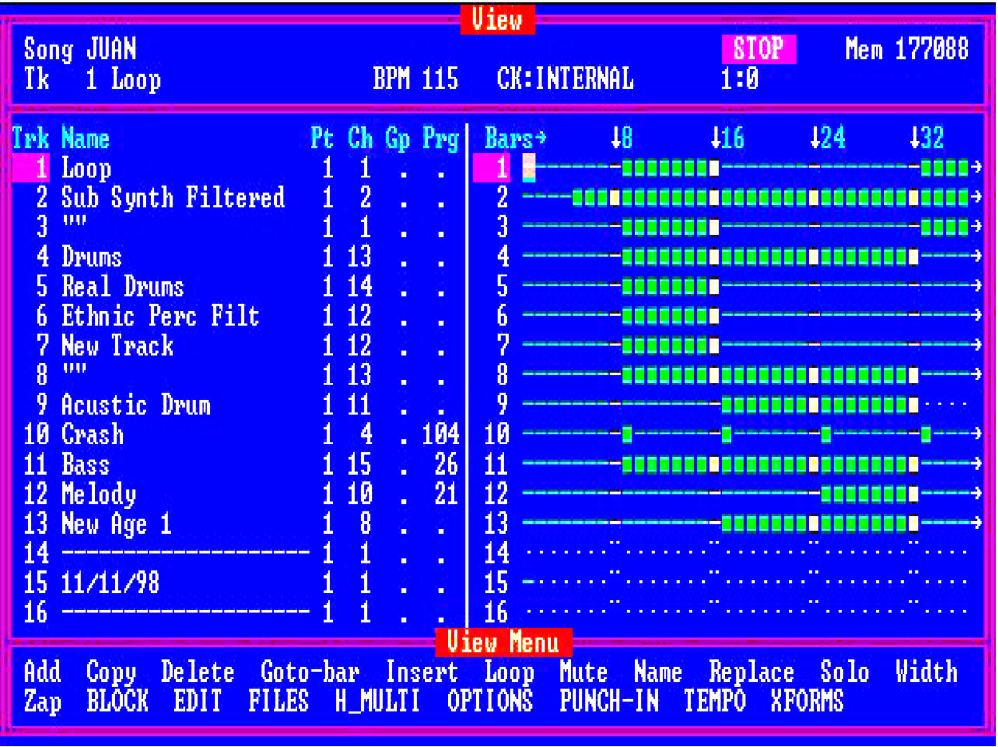


Figure 3.12: Sequencer Plus Gold ("SPG") by Voyetra.

What goes in the AUDIOT archive is a music format called IMF²¹. As it supports only the YM3812, it is tailored for the chip with zero abstraction layers. It consists of a stream of machine language commands with associated delays²².

The stream pilots the nine channels in the OPL2. A channel is able to simulate an instrument and play notes thanks to two oscillators, one playing the role of a modulator and the other the role of a carrier. There are many other ways to control a channel such as envelope, frequency or octave.

The way a channel is programmed is described in detail in Section 4.8.5.1, "OPL2/YM3812 Programming" on page 222.

²¹Id software Music File.
²²IMF format is explained in detail on page 222

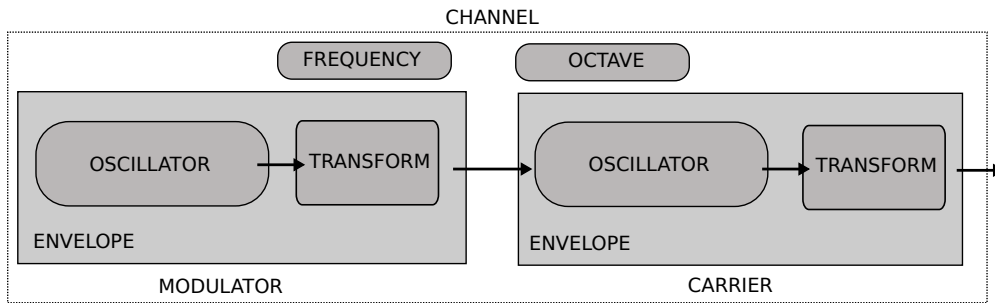


Figure 3.13: Architecture of a YM3812 channel.

Trivia : The YM3812's unmistakable sonority is due to its peculiar set of waveform transformers (they are right after the output of each oscillator in the drawing). Four waveforms are available on the **OPL2**: Sin ①, Abs-sin②, Pulse-sin ③, and Half-sin ④.

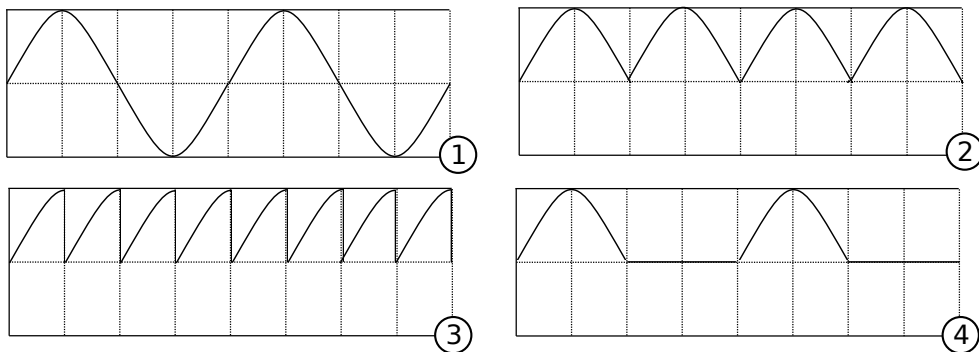


Figure 3.14: The four waveform transforms available.

Trivia : In Episode 3 the music playing features a hidden Morse code message. "To Big Bad Wolf. De Little Red Riding Hood. Eliminate Hitler. Imperative. Complete mission within 24 hours. Out." The end boss of this episode is indeed Hitler in a Mech suit (see hint manual drawing on page 88).

3.7 Distribution

At 4am on May 5th, 1992 the first episode of the game was uploaded to **Massachusetts** via **Software Creations'** **BBS**²³ server. **Wolfenstein 3D** was distributed as shareware; the

²³Bulletin Board Systems were servers that allowed users to connect via a **console** and upload/download programs.

game engine and first episode were given for free and encouraged to be copied and distributed to a maximum number of people. To receive the five other episodes, each player had to pay \$50 to id Software.

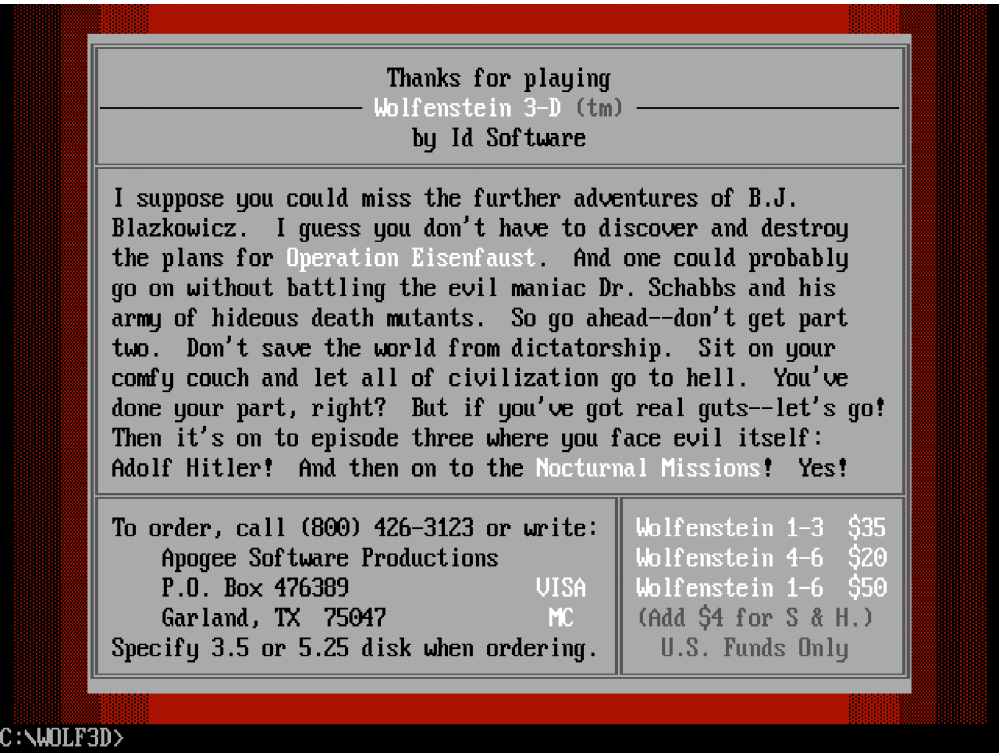


Figure 3.15: Exiting the game describes how to get the full version.

In order to maximize income, it was therefore paramount to make the game easy to copy and redistribute. In 1991, the Internet was still in its infancy and the best medium was the 3½-inch floppy disk. Particular attention was given to the total size of the game so it would fit on one disk. All assets combined accounted for 1,204KiB but everything was compressed to 645KiB (the commonly used low capacity 3½-inch floppy from 1992 could store 720KiB). The full six episodes fit on two disks. The game shipped as follows.

The files can be divided in five parts:

- WOLF3D.EXE: Game engine.
- VSWAP.WL1: Contains all the assets (sprites, textures, digitized sounds) needed during 3D action phases.

- Music and sound effect files used during both 3D and 2D phases:
 - AUDIOHED.WL1 : Index to payload in AUDIOT file.
 - AUDIOT.WL1: Uncompressed audio data.
- Maps:
 - MAPHEAD.WL1 : Index into GAMEMAPS file.
 - GAMEMAPS.WL1 : Compressed Map payloads.
- Pictures used during 2D menu phase:
 - VGAHEAD.WL1 : Index into VGAGRAPH file.
 - VGADICT.WL1 : Huffman-tree to decompress each picture.
 - VGAGRAPH.WL1 : Compressed pics lumped together.

```

C:\WOLF3DU>dir
Directory of C:\WOLF3DU\
.                <DIR>                22-09-2018   9:16
..               <DIR>                22-09-2018   9:17
AUDIOHED WL1      1,156 31-12-1992 23:00
AUDIOT WL1      132,613 31-12-1992 23:00
CONFIG WL1       522 14-09-1994  0:00
FILE_ID DIZ       207 03-12-2011 11:06
GAMEMAPS WL1     27,425 31-12-1992 23:00
MAPHEAD WL1       402 31-12-1992 23:00
VGADICT WL1     1,024 31-12-1992 23:00
VGAGRAPH WL1    326,568 31-12-1992 23:00
VGAHEAD WL1       471 31-12-1992 23:00
USWAP WL1       742,912 31-12-1992 23:00
WOLF3D EXE      109,959 31-12-1992 23:00
    11 File(s)      1,343,259 Bytes.
     2 Dir(s)      262,111,744 Bytes free.

C:\WOLF3DU>_

```

Figure 3.16: All files distributed as shareware as they appear in DOS command prompt.

Trivia : The engine executable is tiny, it only uses 94 KiB on disk since it is LZEXE compressed. The decompressed size is about 280 KiB with 64KiB of it dedicated to the *Signon* screen and 768 bytes for the palette.

The file **extensions** did have meaning:

- WL1: Shareware.
- WL3: Early three-episode full version.
- WL6: Six-episode full version.
- WJ1: Japanese shareware.
- WJ6: Japanese full version.
- SOD: Spear of Destiny.

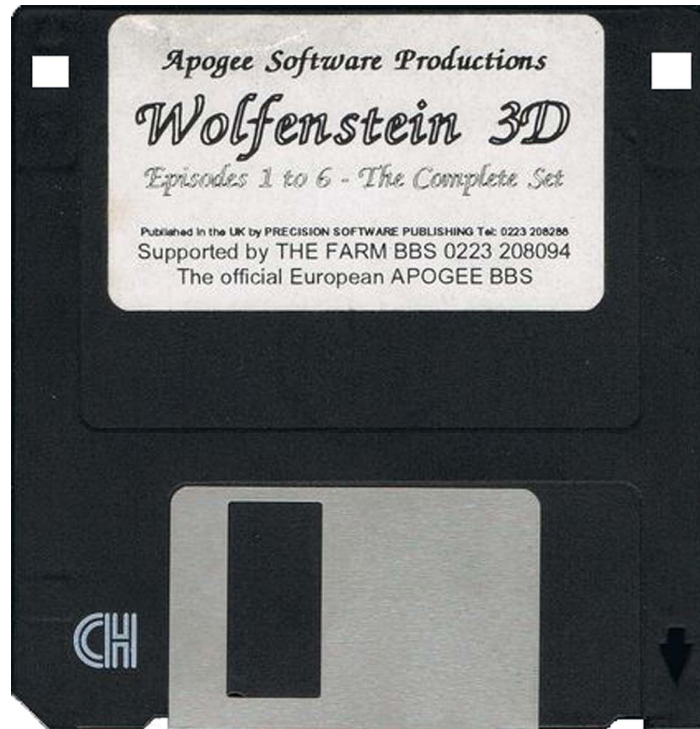


Figure 3.17

Figure 3.17 shows a **3 1/2-inch floppy** widely used to share and redistribute the game. The upper left hole allows the disk reader to recognize if the disk is a low capacity (full = 720 KiB) or a high capacity (hole = 1.44 MiB version). The upper right hole features a sliding tab which when opened forbids writing to the disk, making it read-only. Once inserted in

the floppy disk reader, the large metallic tab slides to the right to expose the magnetic disk.

Trivia : Some people (including the author) saved money by drilling holes in their 720 KiB certified floppy disks to have them recognized as 1.44 MiB. It worked!

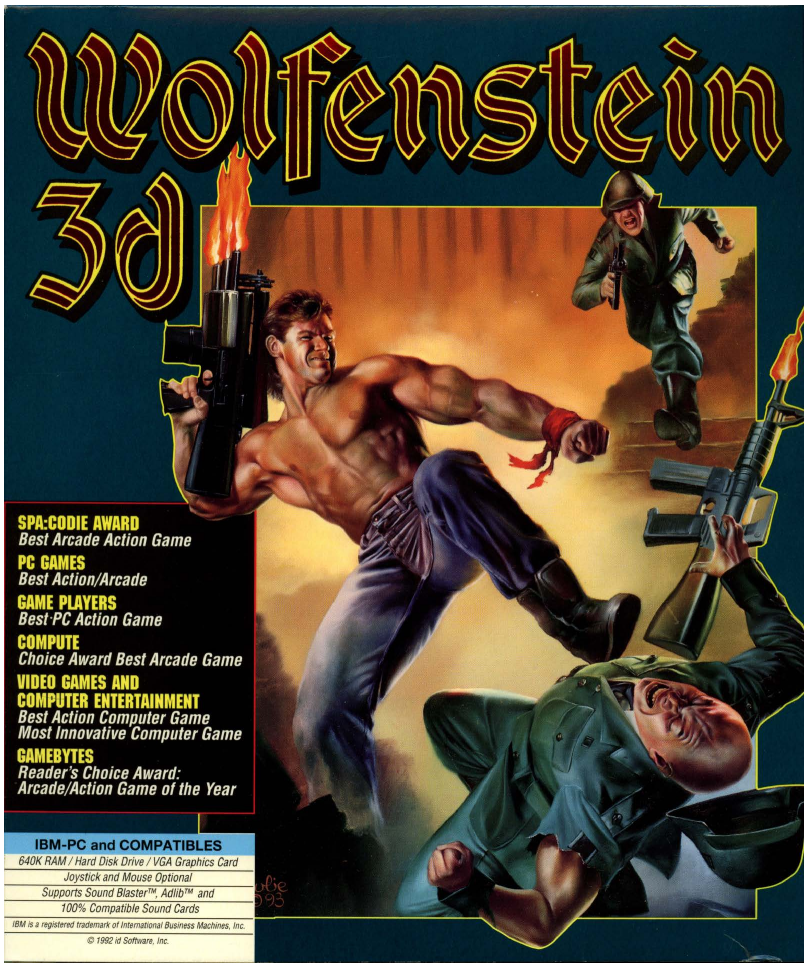


Figure 3.18: Wolfenstein 3D brick and mortar box.

Even though its was originally distributed exclusively as a shareware, sales later called for a more "traditional" medium. GT Interactive picked up Wolfenstein 3D in 1993 and gifted it with an actual box, along with the "Official Hint Manual" which was done by Tom Hall and Kevin Cloud. In 1995, the game was re-released on CD-ROM.

Chapter 4

Software

4.1 Getting the Source Code

The game engine source code was uploaded on [id Software's ftp](#)¹ server on July 21st, 1995.

```
ftp://ftp.idsoftware.com/idstuff/source/wolfsrc.zip
```

More than twenty two years later the archive is still located at the exact same URL, a remarkable fact given the ever changing nature of the web.

Alternatively, you can use [github.com](#) where id Software migrated all of its opensource code around 2012. It is not only faster, it is also more reliable.

```
$ git clone git@github.com:id-Software/wolf3d.git
```

4.2 First Contact

Once downloaded and decompressed, the archive `wolfsrc.zip` contains another self-extracting [PKZIP archive](#). It was a convenience back in the day, but is not practical now and is easy to deflate.

```
$unzip WOLFSRC.1
```

[cloc.pl](#) is a tool which looks at every file in a folder and gathers statistics about source code. It helps for getting an idea of what to expect.

¹File Transfer Protocol.


```
$ cloc -1.64.pl WOLF3D

96 text files.
94 unique files.
27 files ignored.

-----
Language          files          blank          comment          code
-----
C++                26            5750            6201            21169
C/C++ Header       42            802             660             3900
Assembly           10            669             732             2150
DOS Batch           1              1              0                4
-----
SUM:                79            7222            7593            27223
-----
```

The code is 90% in C with assembly² for bottleneck optimizations and low-level I/O such as video or audio³.

Lines of code (SLOC) is not a meaningful metric against a single codebase but excels when it comes to extracting proportions. Wolfenstein 3D with its 27,223 SLOC is very small compared to most software. `curl` (a command-line tool to download url content) is 154,134 SLOC. [Google's Chrome](#) browser is 1,700,000 SLOC. [Linux kernel](#) is 15,000,000 SLOC.

The source code release was overwhelmingly well received yet among the many praises some extraordinary comments left ever-lasting memories.

“ We didn't have spell checkers in our editors back then, and I always had poor spelling. The word "column" appears in the source code dozens of times. After I released the source code, one of the emails that stands out in memory read:

It's "COLUMN", you dumb FUCK!

John Carmack - Programmer ”

²All the assembly in Wolf3D is done with TASM (a.k.a Turbo Assembler by Borland). It uses Intel notation where the destination is before the source: `instr dest source`.

³id Software would not switch to C++ until Doom 3 around 2000.

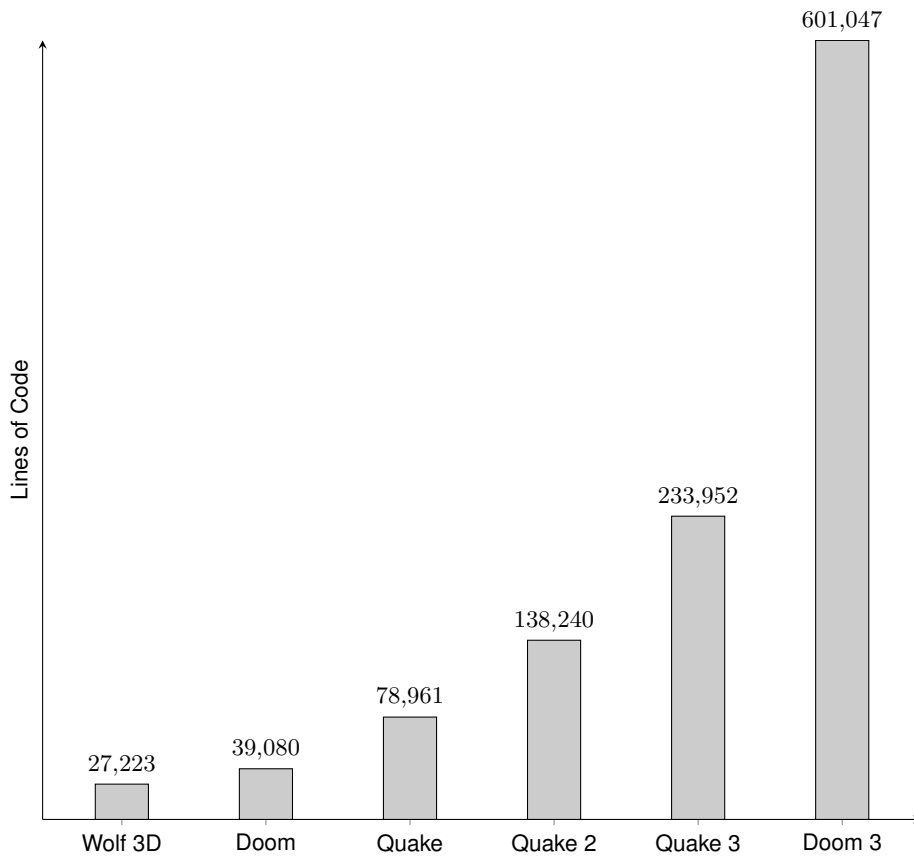


Figure 4.1: Lines of code from id Software game engines.

The archive contains more than just source code; it also features:

- `GOODSTUF.TXT`: Two emails from fans (an ex-POW and a Microsoft employee) demonstrating the success of the game.
- `SIGNON.OBJ`: The startup screen showing the system characteristics (RAM, EMS, XMS, Joystick, SoundCards) was linked in the binary. This weird design choice is explained later.
- `GAMEPAL.OBJ`: Game palette. Hardcoded and linked in the executable for the same reason as `SIGNON.OBJ`.
- `README`: How to build. You can also find a complete tutorial in "Let's compile like it's 1992" on fabiensanglard.net.
- Many files resulting from a previous compilation attempt.

4.3 Big Picture

The game engine is divided in three blocks:

- 2D menu engine which lets users configure the game.
- 3D game renderer where the users spend most of their time.
- Sound system which runs concurrently with either the 2D or 3D renderer.

The three systems communicate via shared memory. The renderer writes music and sound requests to the RAM (also making sure the assets are ready). These requests are read by the sound "loop". The sound system also writes to the RAM for the renderers since it is in charge of the heartbeat of the whole engine. The renderers update the world according to the wall-time tracked by `TimeCount` variable.

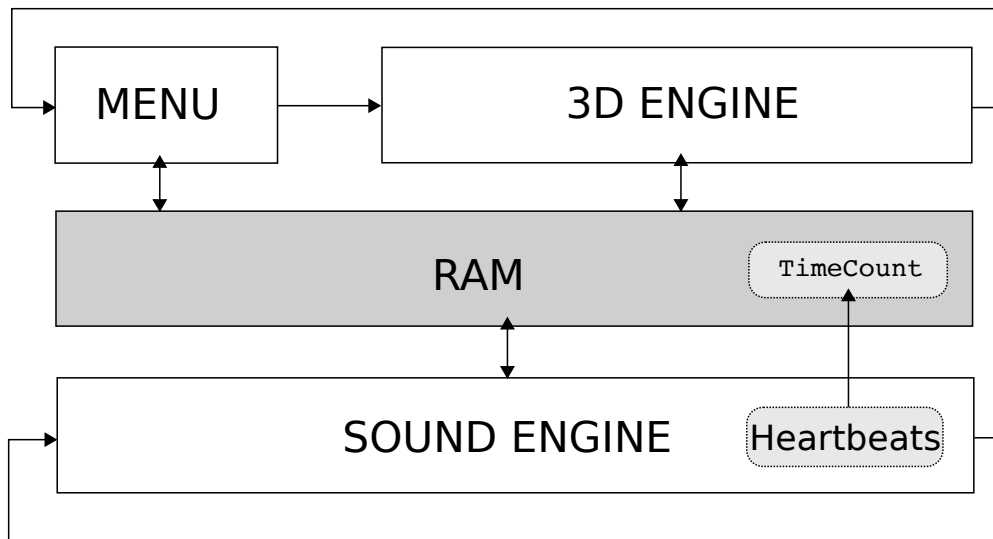


Figure 4.2: Game engine three main systems.

4.3.1 Unrolled Loop

With the big picture in mind, we can dive into the code and unroll the main loop starting in `void main()`. The two renderers are regular loops but due to limitations explained later, the sound system is interrupt-driven and therefore out of `main`. Because of **real mode**, C types don't mean what people would expect from a 32-bit architecture.

- `int` and `word` are 16 bits.
- `long` and `dword` are 32 bits.

The first thing the program **does** is check the assets available via `CheckForEpisodes()`.

```
void main (void) {
    CheckForEpisodes();
    Patch386 ();
    InitGame ();
    DemoLoop();
}
```

Since the target machine ran in real mode, the code was compiled using 16-bit instructions only. For operations on long (32 bit), Borland used its own **math library**. In `Patch386` *Wolfenstein 3D* detects if the CPU is a 386 and patches its own code to replace Borland's integral division with instructions using 32-bit registers `eax` and `edx`.

```
mov eax,[bp+8]
cdq
idiv [DWORD PTR bp+12]
mov edx, eax
shr edx,16
```

In `InitGame`, the engine starts up and brings up all the managers.

```
void InitGame () {
    MM_Startup ();           // Memory manager

    SignonScreen ();        // Show system configuration

    VW_Startup ();          // Video Manager
    IN_Startup ();          // Input Manager
    PM_Startup ();          // Page Manager
    PM_UnlockMainMem ();
    SD_Startup ();          // Sound manager
    CA_Startup ();          // Cache manager
    US_Startup ();          // Font manager
    InitDigiMap ();
    ReadConfig ();
    CA_CacheGrChunk(STARTFONT); // Load font
    MM_SetLock (&grsegs[STARTFONT],true); // Lock font
    LoadLatchMem ();        // Load picture asset to VRAM
    BuildTables ();          // sin/cos/view lookup tables
    SetupWalls ();          // Lookup table wall textures
}
```

Then comes the core loop, where the 2D renderer and 3D renderer are called forever.

```

void DemoLoop() {
    StartCPMusic(INTROSONG);
    PG13(); // Show Profound Carnage screen
    while (1) {
        CA_CacheScreen (TITLEPIC);
        CA_CacheScreen (CREDITSPIC);
        DrawHighScores ();
        PlayDemo (0);
        GameLoop (); // 2D renderer (menu)
        SetupGameLevel ();
        StartMusic ();
        PM_CheckMainMem ();
        PreloadGraphics ();
        DrawLevel ();
        PlayLoop () ; // 3D renderer (action)
        StopMusic ();
    }
    Quit("Demo loop exited???");
}

```

PlayLoop contains the 3D renderer. It is pretty standard with getting inputs, update world, and render world approach.

```

void PlayLoop () {
    PollControls (); // Get player input

    MoveDoors (); // Move doors
    MovePWalls (); // Move secret wall
    for (obj = player; obj; obj = obj->next)
        DoActor (obj); // Enemies think

    ThreeDRefresh () { // Render 3D view
        VGAClearScreen (); // Draw floor/ceiling
        WallRefresh (); // Draw walls
        DrawScaleds(); // draw scaled stuff
        DrawPlayerWeapon (); // draw weapon
        [...] // Flip framebuffer via CRT Controller
    }
    UpdateSoundLoc (); // Stereo sound loc
}

```

The sound system is started via the Sound Manager in `SDL_SetTimerSpeed`. While there is a famous game development library called Simple DirectMedia Layer (SDL), the prefix `SDL_` has nothing to do with it. It stands for Sound Low level (Simple DirectMedia Layer

did not even exist in 1991).

The reason for interrupts is extensively explained in Chapter 4.8 "Audio and Heartbeat". In short, with an OS supporting neither processes nor threads, it was the only way to have something execute concurrently with the rest of the engine.

An ISR (Interrupt Service Routine) is installed in the Interrupt Vector Table to respond to interrupts triggered by the engine. Note how the ISR can be called at frequencies of 140Hz, 700Hz, or even 7000Hz depending on the needs of the sound system.

```
#define TickBase 70

typedef enum {
    sds_Off,
    sds_PC,
    sds_SoundSource,
    sds_SoundBlaster
} SDSMode;

extern SDSMode DigiMode;

static void SDL_SetTimerSpeed(void) {
    word rate;
    void interrupt (*isr)(void);

    if ((DigiMode == sds_PC) && DigiPlaying) {
        rate = TickBase * 100;    // 7000 Hz
        isr = SDL_t0ExtremeAsmService;
    }
    else if (music || ((DigiMode == sds_SoundSource)
        && DigiPlaying)) {
        rate = TickBase * 10;    // 700 Hz
        isr = SDL_t0FastAsmService;
    }
    else {
        rate = TickBase * 2;    // 140 Hz
        isr = SDL_t0SlowAsmService;
    }
    setvect(8, isr);
    SDL_SetIntsPerSec(rate);
}
```

4.4 Architecture

The source code is structured in two layers. WL_* files are high-level layers relying on low-level ID_* sub-systems called Managers interacting with the hardware.

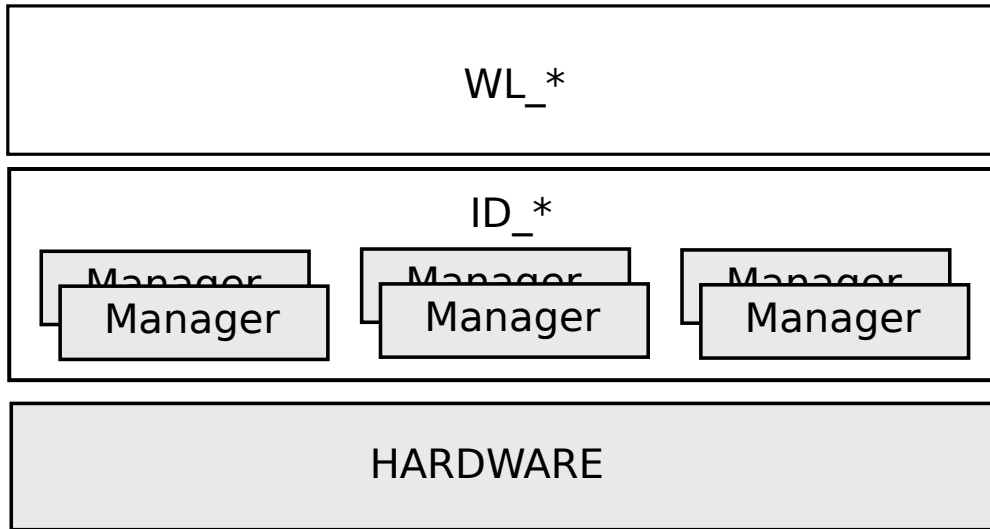


Figure 4.3: Wolfenstein 3D source code layers.

There are seven managers in total:

- Memory
- Page
- Video
- Cache
- Sound
- User
- Input

The WL_ stuff was written specifically for Wolf3D while the ID_ managers were reused from previous games (Hovortank One and Catacomb 3-D) and improved for the needs of the new engine.

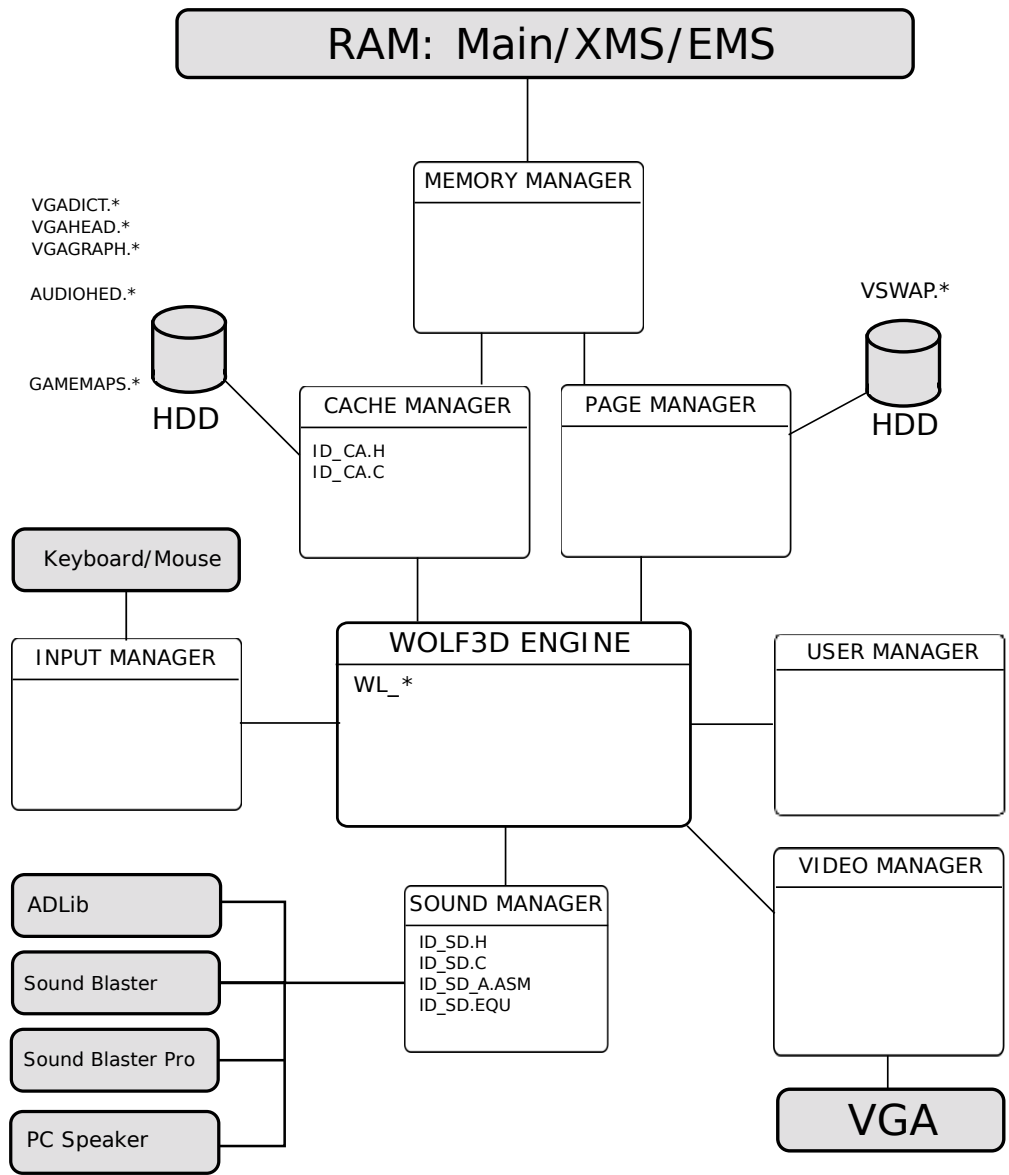


Figure 4.4: Architecture with engine and sub-systems (in white) connected to I/O (in gray).

Next to the hard drives (HDD) you can see the assets packed as described in Chapter 3 Team.

4.4.1 Memory Manager (MM)

The engine does not rely on `malloc` to manage conventional memory, as this can lead to fragmented memory and no way to compact free space. It has its own memory manager made of a linked list of "blocks" keeping track of the RAM. A block points to a starting point in RAM and has a size.

```
typedef struct mmblockstruct
{
    unsigned    start,length;
    unsigned    attributes;
    memptr      *useptr;
    struct mmblockstruct far *next;
} mmblocktype;
```

A block can be marked with attributes:

- LOCKBIT : This block of RAM cannot be moved during compaction.
- PURGEBITS : Four levels available, 0= unpurgeable, 1= purgeable, 2= not used, 3= purge first.

The memory manager starts by allocating all available RAM via `malloc/farmalloc` and creates a LOCKED block of size 1KiB at the end. The linked list uses two pointers: `HEAD` and `ROVER` which point to the second to last block.

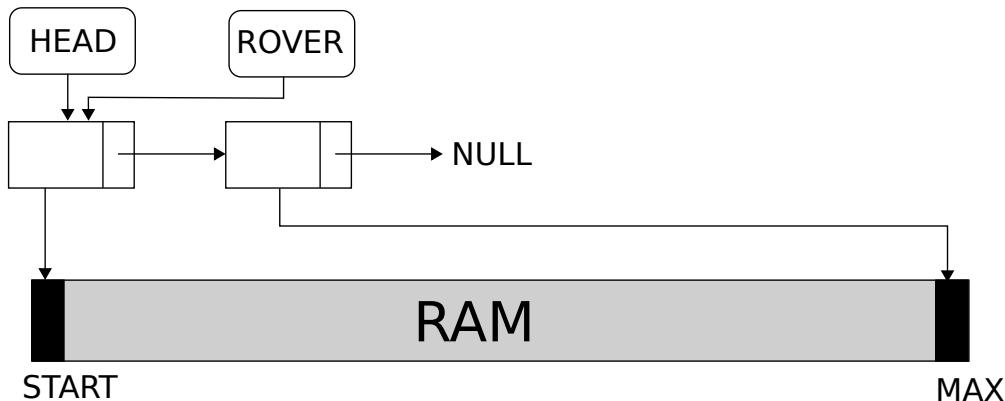


Figure 4.5: Initial memory manager state.

The engine interacts with the Memory Manager by requesting RAM (`MM_GetPtr`) and freeing RAM (`MM_FreePtr`). To allocate memory, the manager searches for "holes" between blocks. This can take up to three passes of increasing complexity:

1. After rover.
2. After head.
3. Compacting and then after rover.

The easiest case is when there is enough space after the rover. A new node is simply added to the linked list and the rover moves forward. In the next drawing, three allocation requests have succeeded: A, B and C.

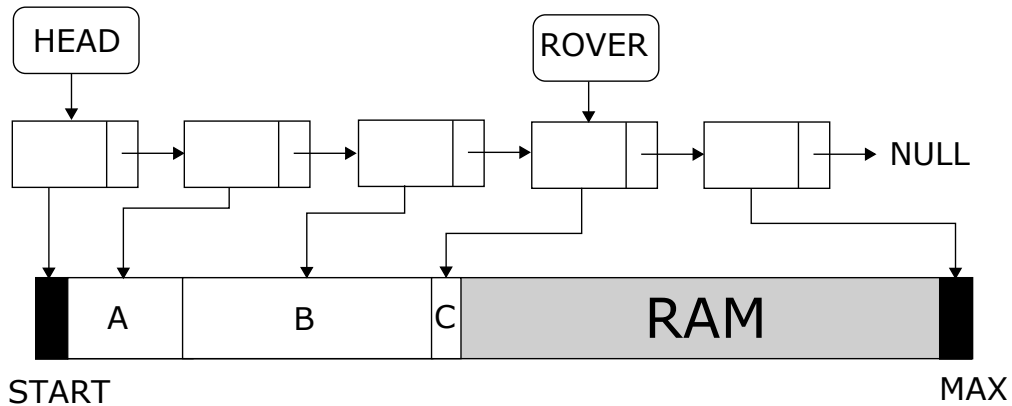


Figure 4.6: MM internal state after three pass 1 allocations.

Eventually the free RAM will be exhausted and the first pass will fail.

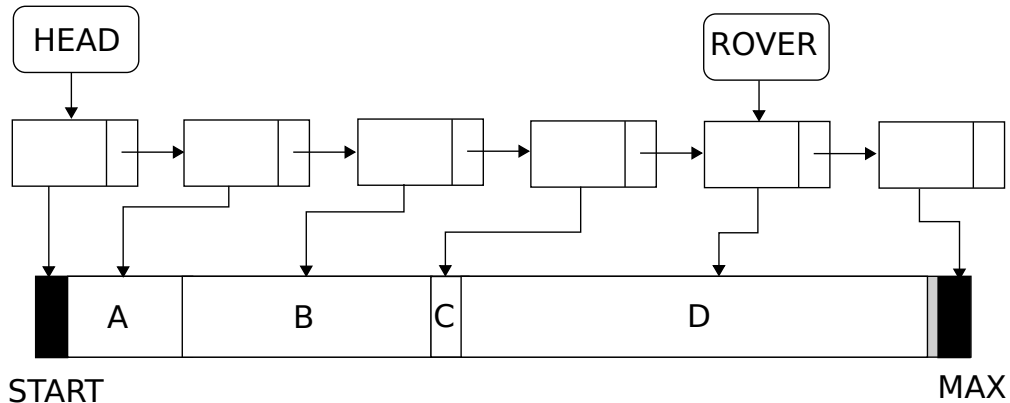


Figure 4.7: Pass 1 failure: Not enough RAM after the ROVER.

If the first pass fails, the second pass looks for a "hole" between the head and the rover. This pass will also purge unused blocks. If for example block B was marked as PURGEABLE, it will be deleted and replaced with the new block E. At this point fragmentation starts to appear (like if `malloc` was used).

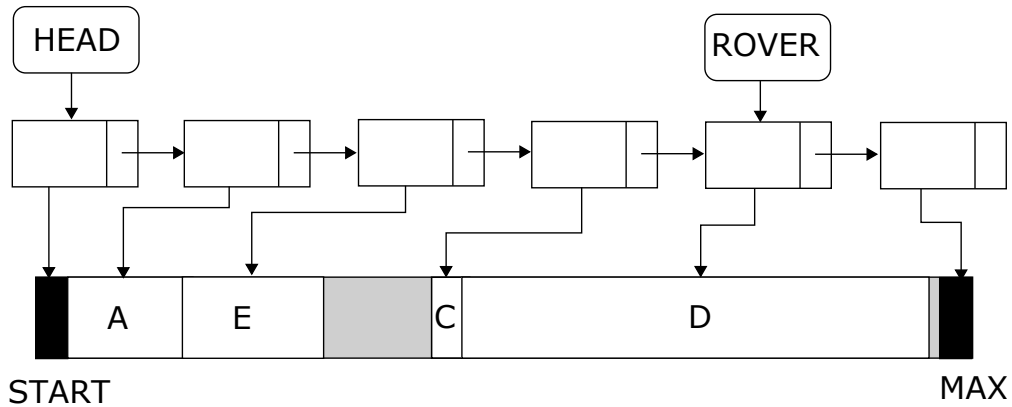


Figure 4.8: B was purged. E was allocated in pass 2.

If the first and second pass fail, there is no continuous block of memory large enough to satisfy the request. The manager will then iterate through the entire linked list and do two things: delete blocks marked as purgeable, and compact the RAM by moving blocks.

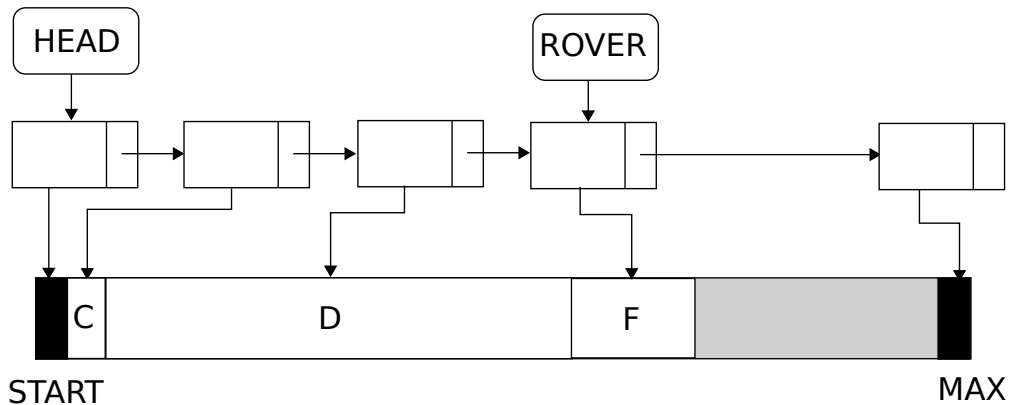


Figure 4.9: A and E were purged. C and D compacted. F allocated in pass3.

But if memory is moved around, how do previous allocations still point to what they did before the compaction phase? Notice that a `mmblockstruct` has a `useptr` pointer which

points to the owner of a block. When memory is moved, the owner of the block is also updated.

As some blocks are marked as `LOCKED`, compacting can be disturbed. Upon encountering a locked block, compacting stops and the next block will be moved immediately after the locked block, even if there was space available between the last block and the locked block.

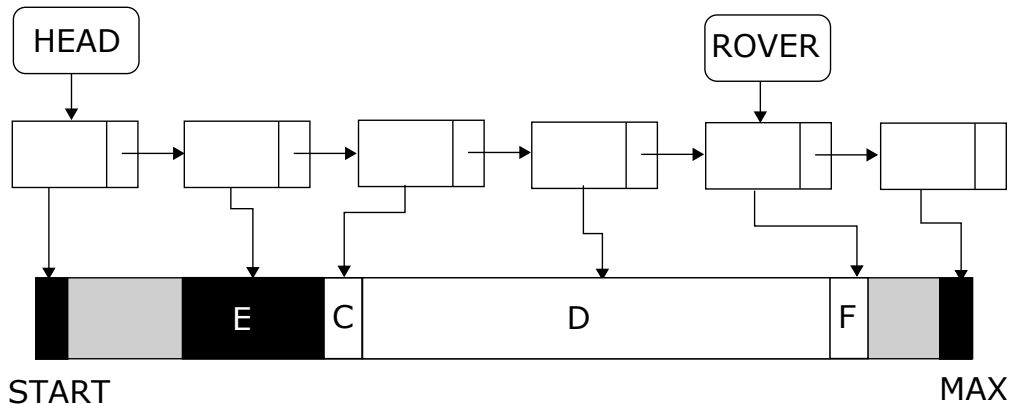


Figure 4.10: E is locked and cannot be compacted.

In the above drawing, C was moved after E, even though it could have been moved before. Avoiding this waste would have made the memory manager more complicated, so the waste was deemed acceptable. Often in designing a component you have to be practical and establish a certain trade off between accuracy and complexity.

“

A dedicated memory manager was probably justified for Wolf, but they are a huge source of bugs, and I urge people not to do it!

John Carmack - Programmer

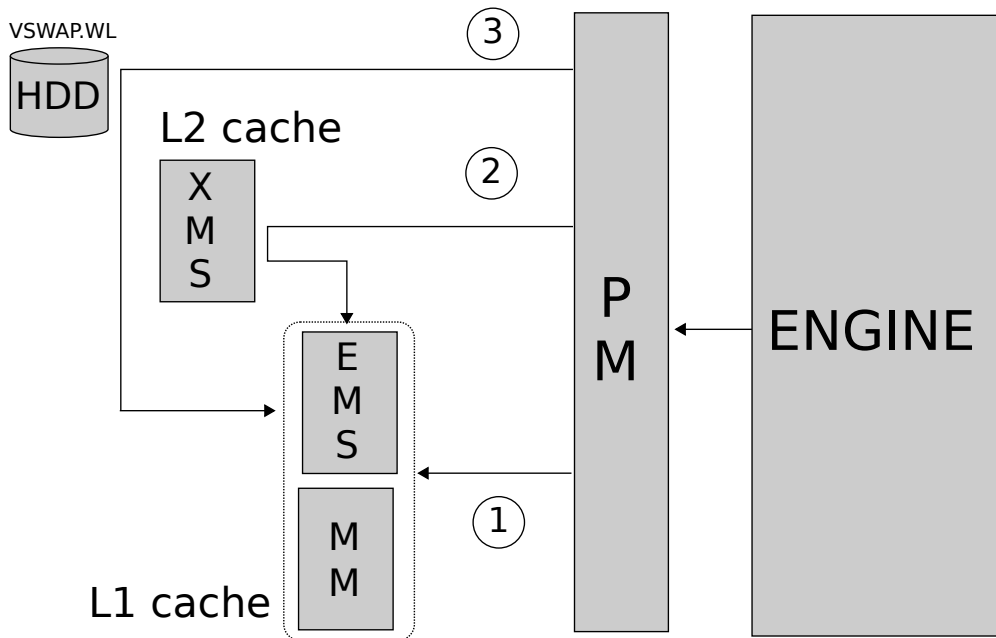
”

4.4.2 Page Manager (PM)

The Page Manager is dedicated to the 3D engine. Its task is to make sure assets such as wall textures, sprites, and sound effects stored on HDD are available in RAM for the CPU to use. Jason Blochowiak seems to have been the main author and his previous experience with Unix clearly influenced the design of this component. It is built around the concept of paging and swapping.

Instead of using a memory address to identify a page like Unix, an asset ID is used. These IDs are generated by IGRAB. Each asset consumes a full "page". Like Unix, all pages have the same size, 4KiB. When the engine needs a resource, it requests a page with the resource ID from the Page Manager. All types of RAM (Conventional, EMS, and XMS) are leveraged but there is a hierarchy.

Originally all assets for 3D sequences are on HDD in file `VSWAP.WL1`. When a request for an asset is received, the L1 cache (comprised of Conventional and EMS RAM) is looked up first ①. In case of a miss, the L2 cache is consulted ②. If the page is found there, it is transferred to L1. If the page is still not found in L2, it is loaded from the HDD directly to L1 ③. L2 is only written to when a page is evicted from L1. Every time a page is accessed, it is also tagged with the current frame number. This tag is used to enforce the eviction policy.



The architecture of the Page Manager is interesting since it treats XMS as a last resort L2 cache level while EMS RAM is used like conventional memory. This is because EMS driven RAM is several times faster than XMS driven RAM and almost as fast as conventional memory. This topic is detailed in Appendix B on page 297.

In order to minimize the cost of page misses, the engine preloads the page cache before a level starts. The user experiences this as the "Get Psyched" screen:



Figure 4.11: The "thermometer" showing the Page Manager precaching process.

The precaching mechanism is not particularly clever: it loads as many pages from the swap file as possible. It doesn't try to look at what is actually used in the level but instead loads assets in the order they are stored in VSWAP file on the HDD. On a machine with low memory (less than 1MB), the eviction policy (LRU) stabilizes the cache after a few minutes in a level.

This design has a small yet annoying flaw. On a low memory machine, a cache miss will occur when the player opens the last door of a level and is about to find herself confronted with the heavy-power final boss. This enemy has never been seen before, and therefore is not in the cache of the Page Manager. This incurs the worst possible case of cache miss: a long access to the hard-drive is required, leading to lag and often resulting in an undeserved (and humiliating) death.

The size of the swap file varies depending on the version of the game being played: VSWAP.WL1 (shareware) is 742KiB while VSWAP.WL6 (full version) is 1,500KiB. In both cases a machine with 2MiB of RAM on top of the factory-issued 1MiB is enough to have

all assets loaded during precaching.

Thrashing: When the system has to evict pages but ends up reloading the same resource during the same frame, "thrashing" occurs. The HDD is put to heavy use, and framerate drops. Thrashing can happen if too many different resources are visible on the screen. In order to help designers balance their creativity with the need for a decent framerate, the engine detects thrashing and flashes the screen border red when running in dev-mode.

Sounds are special: Because sound cards are fed via a system of interrupts, the sound manager cannot recover from a page miss. Therefore, all sound resources are loaded first (they are located at the start of VSWAP.WL1) and only in conventional memory.

4.4.3 Video Manager (VL & VH)

The video manager features two parts. There is a higher level component and a lower level one:

- The VL_* layer is made of both C and ASM, where the C functions abstract away VGA register manipulation via assembly routines.
- The high-level layer (VH_*) is dedicated to 2D menu drawings and is of course a client of the lower layer.

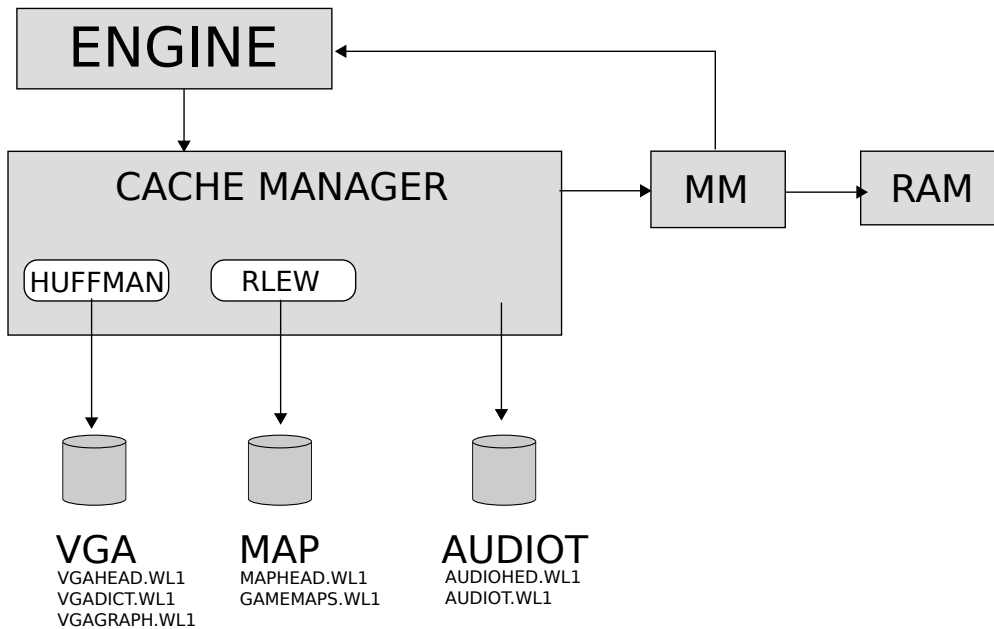
4.4.4 Cache Manager (CA)

The cache manager is a small but critical component. It loads and decompresses maps, 2D graphics, and audio resources stored on the filesystem and makes them available in RAM. Assets of each kind are stored in two files. A header file contains the offset to allow translation from asset ID to byte offset in the data file.

All resources are compressed. In the case of maps and audio, compression is hard-coded in the engine. For graphics, however, a third file (DICT) contains the compression dictionary to decompress each asset. The Cache Manager handles decompression transparently, relying on two compression methods. Assets use a traditional huffman method but the maps are compressed twice with a "carmacized" pass which is an independent re-discovery of the LZ (Lempel-Ziv) approach⁴.

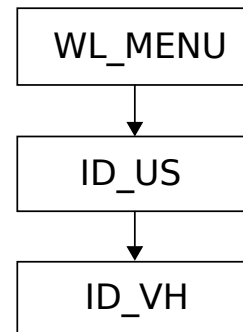
Trivia : Is there a typo in the filename AUDIOHEAD.WL ? The correct spelling would be AUDIOHEAD.WL1. Where is the A? This is in fact a limitation of the operating system. DOS only allows 8.3 filenames (at most eight characters followed by a dot, then at most three characters for the extension).

⁴John Carmack mentioned on several occasions how difficult it was to access programming books back in the '90s. He would "invent" something only to find out later others had already done it and better than him!



4.4.5 User Manager (US)

The user manager is largely based on the Catacomb 3-D code and was written by Jason Blochowiak. The copy/paste is very visible since 90% of the functions declared in the header (ID_US.H) are not actually implemented in ID_US.C. It is a poorly named manager as it mostly takes care of text layout. When a WL_* high-level routine needs to draw a string, it is passed to US_Print which does all measurement (e.g. Draw string centered) and then passes this information to the Video Manager (VW_DrawPropString), which takes care of rendition.



4.4.6 Sound Manager (SD)

The Sound Manager abstracts interaction with all four sound systems supported: PC Speaker, AdLib, Sound Blaster, and Disney Sound Source. It is a beast of its own since it doesn't run inside the engine. Instead it is called via IRQ at a much higher frequency than the engine (the engine runs at a maximum 70Hz, while the sound manager ranges from 140Hz to 7000Hz). It must run quickly and is therefore not only written in assembly, but its assets are also privileged when it comes to memory allocation. All of its assets are loaded

in conventional memory to avoid a cache miss in the page manager.

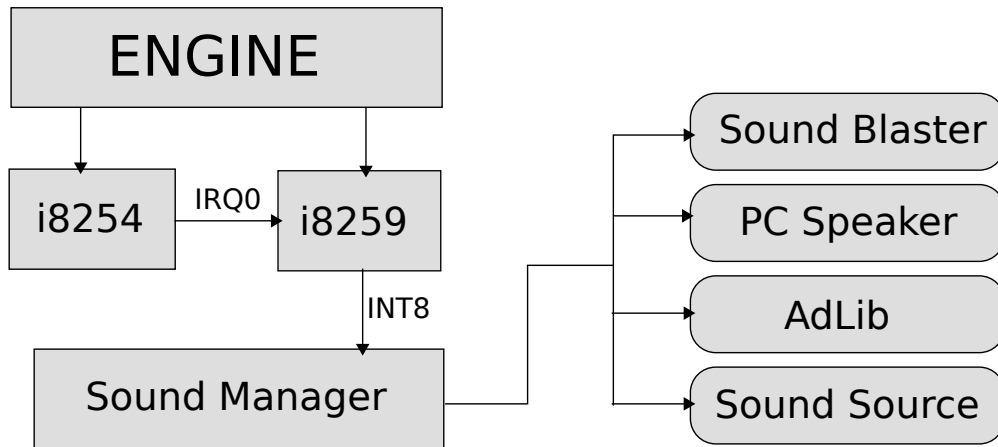


Figure 4.12: Sound system architecture.

The sound manager is described extensively in the "Sound and Music" section.

4.4.7 Input Manager (IN)

The input manager abstracts interactions with joystick, keyboard, and mouse. It features the boring boilerplate code to deal with PS/2, Serial, and DA-15 ports, with each using their own I/O addresses.

4.5 Startup

As the game engine starts, it must deal with the difficulties described in Chapter 2 "Hardware". This is where things become really interesting.

4.5.1 Signon

The first (mild) issue to deal with was the heterogeneous ecosystem of PCs on the market. With different drivers loaded and different sound cards, the engine has to figure out how much RAM is installed and whether or not it will be able to run. If not, it has to let the user know what the problem is. That was important as id Software was a small team, without the resources to help troubleshoot customer issues.

That is what the "signon" screen is for: self diagnostics.



Figure 4.13: Signon screen.

Besides showing recognized devices such as mouse, joystick, and sound cards, the signon screen's most important metric is labeled "MAIN". Due to the architecture described in Chapter 2 "Hardware", a DOS program has only 640KiB of RAM available. Each driver loaded by the user takes away from these 640KiB. If DOS cannot load the executable in RAM, the user will see the following error message:

```
Out of memory
```

The signon screen shows that Wolf3D needs at least 320KiB of Conventional RAM. John Romero wrote a release note to help people understand what was going on and avoid angry calls. You can read it in the Appendix under "The 640KB Barrier".

At the time signon is displayed, the only loaded manager is the Memory Manager. There is not even a filesystem for the engine to access yet. That is why the palette and the signon screen are compiled within the executable. This is done so they are loaded in RAM by the operating system (DOS) loader. All the engine does is load the palette into the VGA,

copy the signon bitmap from RAM to VRAM, and fill the "thermometer" blocks with green or yellow based on what was detected.

```
// Content of GAMEPAL.OBJ:
// account for 256 * 3 = 768 bytes.
extern byte far gamepal;

// Content of SIGNON.OBJ:
// accounts for 320x200 = 64,000 bytes.
extern char far introscn;

void SignonScreen (void)
{
    unsigned    segstart, seglength;

    VL_SetVGAPlaneMode ();
    VL_TestPaletteSet ();
    VL_SetPalette (&gamepal);

    if (!virtualreality)
    {
        VW_SetScreen(0x8000,0);
        VL_MungePic (&introscn,320,200);
        VL_MemToScreen (&introscn,320,200,0,0);
        VW_SetScreen(0,0);
    }

    // reclaim the memory from the linked signon screen
    segstart = FP_SEG(&introscn);
    seglength = 64000/16;
    if (FP_OFF(&introscn)){
        segstart++;
        seglength--;
    }
    MML_UseSpace (segstart, seglength);
}
```

After that screen, the introscn variable (using 320x200 bytes = 64,000 bytes) is unloaded from RAM to make more room for runtime.

Trivia : The body of function SignonScreen features a reference to "virtual reality" which was added for a company licensing the engine to build arcade VR cabinets. Several decades later, John Carmack would be involved in a VR renaissance with Oculus VR.

4.5.2 Solving the VGA Problem

Chapter 2 "Hardware" left us with an unresolved issue: all the VGA modes lack double buffering capability.

The most appealing mode (13h) offers a single framebuffer at a resolution of 320x200 non-square pixels with 256 indexed colors. The Chain-4 chipset in the VGA circuitry automatically maps the RAM starting at A0000h to the four VRAM banks. When active, a developer needs not to worry about banks. The screen can be cleared with a simple function as follows.

```
char far *VGA = (byte far*)0xA0000000L;

void ClearScreen(void){
    asm mov ax,0x13
    asm int 0x10

    for (int i=0 ; i < 320*200 ; i++)
        VGA[i] = 0x00;
}
```

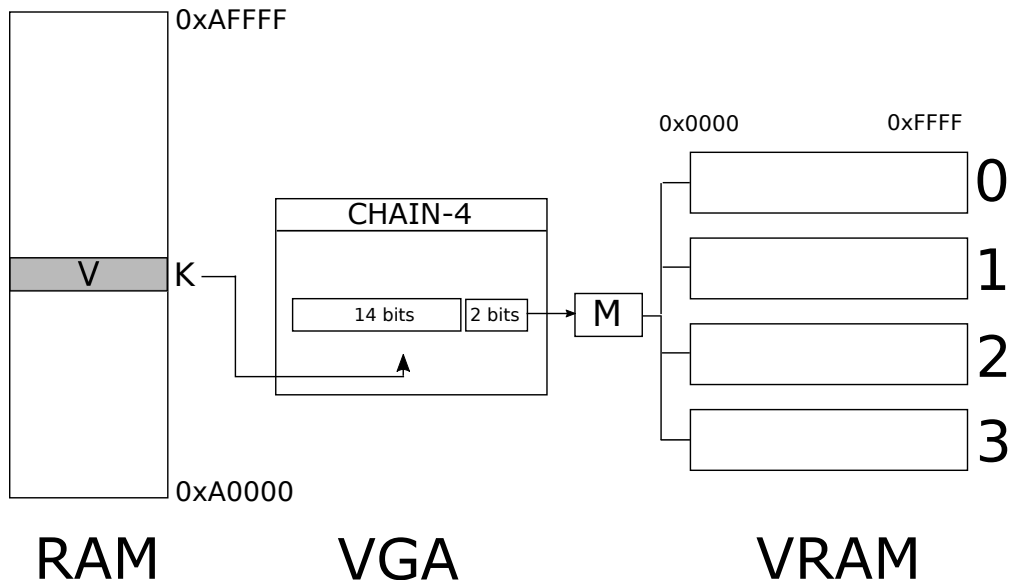


Figure 4.14: Chain-4 chipset between RAM and VRAM routes I/Os operations.

This mapping system is also called "chaining". Because 2 bits out of the address are used

to route a write/read operation to a bank, only 14 bits are used for the actual offset in the bank. Since 14 bits can only address 16384 values, this system results in 75% of VRAM left unusable.

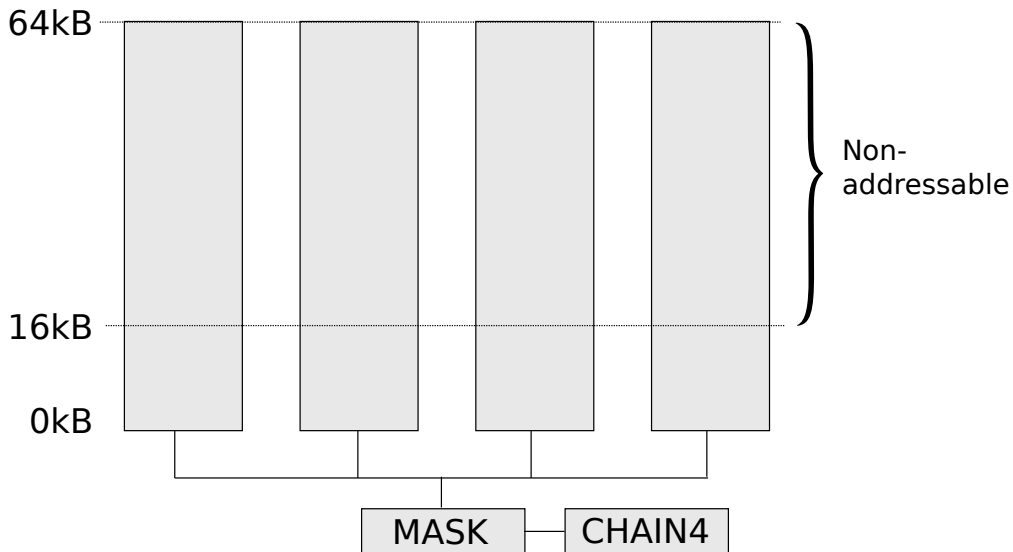


Figure 4.15: Chain-4 fakes one continuous VRAM bank but wastes 75% of the VRAM.

The waste is really the fault of the Chain-4 chip. However it turns out it is possible to disable this. The technique was popularized by Michael Abrash in *Dr. Dobbs's Journal* of July 1991. In his article he described what he coined Mode-X. An undocumented sequence of operations disabling Chain-4 allows for a resolution of 320x240 square pixels (since the ratio is 4:3) and full access to the 256KiB of RAM.

Wolfenstein 3D does things slightly differently. It disables Chain-4 but keeps resolution at 320x200. This mode was coined Mode-Y one year later⁵.

There are two reasons the team did not use 320x240 square pixels. Despite its advantage for artists, a screen using Mode-Y is 320x200 = 64,000 pixels, which represents 17% fewer pixels compared to Mode-X with 320x240 = 76,800 pixels. The engine already struggled to reach an acceptable framerate, so Mode-X was simply too many pixels per frame. It would have also been inconvenient to artists since Deluxe Paint ran in Mode 13h which has non-square pixels. This would have created an awkward pipeline where assets would have been created and rendered at different resolutions.

⁵rec.games.programmer, February 10th, 1992

```

void VL_SetVGAPlaneMode (void) {
    // Call 16th interrupt vector with value 0x13
    // (Ask the BIOS to setup the VGA in mode 13h)
    asm mov ax,0x13
    asm int 0x10

    // Unchain (called deplane in the engine)
    VL_DePlaneVGA ();
    VGAMAPMASK(15);
    VL_SetLineWidth (40);
}

```

The magic happens in function VL_DePlaneVGA where VGA registers are manipulated to tweak what the BIOS had setup in Mode 13h.

```

#define SC_INDEX      0x03c4
#define SC_DATA       0x03c5

#define CRTC_INDEX    0x03d4
#define CRTC_DATA     0x03d5

#define MEMORY_MODE    0x04
#define CRTC_UNDERLINE 0x14
#define CRTC_MODE      0x17

void VL_DePlaneVGA() {
    // Change how VRAM is written (Disable Chain-4)
    outp(SC_INDEX, MEMORY_MODE);
    outp(SC_DATA, (inp(SC_DATA)&~8));

    // Clear all four banks since the bios only cleared
    // the first 16K of each bank when setting up mode 13h.
    VL_ClearVideo (0);

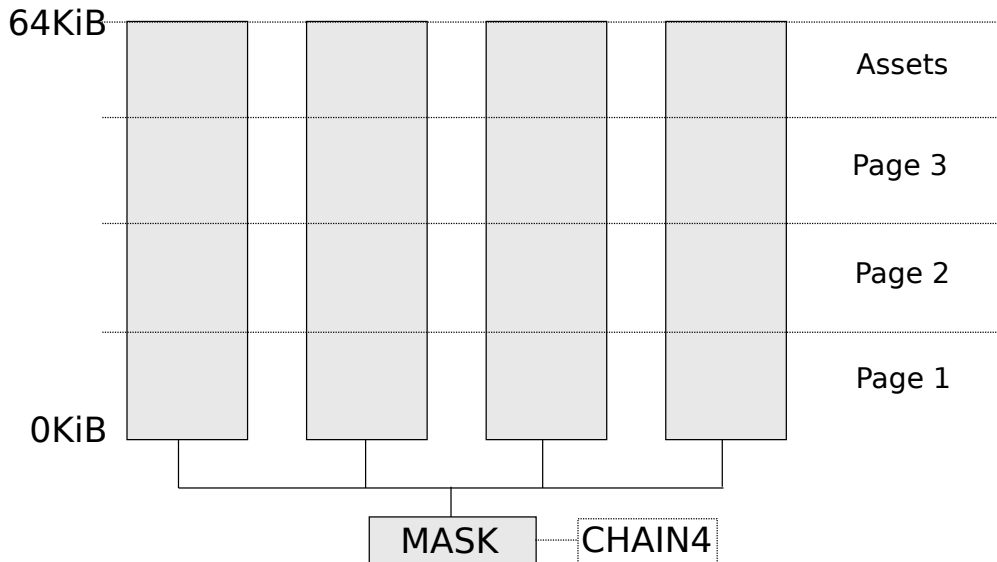
    // Change how VRAM is read by the CRTC
    // Addressing mode is selected via CRTC_MODE register.
    outp(CRTC_INDEX, CRTC_UNDERLINE);
    outp(CRTC_DATA, 0x00);
    // CRTC addressing mode set to byte.
    outp(CRTC_INDEX, CRTC_MODE);
    outp(CRTC_DATA, 0xa3);
}

```

The VGA registers of the Sequence Controller and CRT Controller are setup to divide the

256KiB VRAM into four parts.

- 64,000 bytes for Framebuffer 0
- 64,000 bytes for Framebuffer 1
- 64,000 bytes for Framebuffer 2
- 70,144 bytes for Graphic assets



But the engine is not done yet. Tweaking Mode 13h into Mode-Y solves one big problem but introduces two smaller ones: one regarding speed and one regarding correctness.

Let's look at speed first. With Chain-4 out of the picture the developer is now in charge of selecting the bank to write to. This can easily be done with a simple function.

```
#define SC_MAPMASK 0x02

void selectPlane(char plane) {
    outp(SC_INDEX, SC_MAPMASK);
    outp(SC_DATA, 1 << plane);
}
```

Now the code sample from the hardware chapter on page 59 to clear the screen only adds a modulo.

```

void CleanScreen(int color) {
    for(int y=0 ; y < 200 ; y++) {
        for(int x=0; x < 320 ; x++) {
            selectPlane(x % 4);
            writePixel(x, y, color);
        }
    }
}

```

The code looks innocuous, but as simple as it is, it cannot run at more than a few frames per second⁶. The problem is we replaced something done in hardware with something done in software. The `outp` instructions are simply too slow.

The solution is to change how we draw to the screen. Instead of drawing horizontally first we need to draw vertically first in order to minimize the number of bank switches.

```

void CleanScreen(int color) {
    for(int x=0; x < 320 ; x++) {
        selectPlane(x % 4);
        for(int y=0 ; y < 200 ; y++) {
            writePixel(x, y, color);
        }
    }
}

```

This code runs twice as fast⁷ since only 640 slow `outp` instructions are used⁸.

This speed consideration has a fundamental impact on the engine: to draw anything fast with the VGA, it has to be drawn vertically. Everything in the engine is drawn this way: walls, sprites, menus. The ramifications of this hardware constraint are felt all the way down to how assets are stored in RAM: rotated 90 degrees and woven to match the VGA bank layout. This is described in detail in section 4.7.8 "Drawing Sprites".

The second issue introduced by Mode-Y is about correctness. With three pages available, the engine draws in page 1, then page 2, then page 3, and then goes back to page 1. This solves tearing and allows the engine to never block on vsync since it always has a valid framebuffer to draw to. Changing pages is done by instructing the CRT Controller that it should scan a framebuffer at a different offset after the next vsync.

The CRT Controller scan offset is a 16-bit value which is updated with a little bit of assem-

⁶5 frames per second on a 386DX-40 with Cirrus Logic VGA card.

⁷10 frames per second on a 386DX-40 with Cirrus Logic VGA card.

⁸To reach 70 frames per second is possible by using fewer instructions thanks to `REP STOSW` instruction.

bly writing to a VGA register: first the high byte and then the low byte as follows.

```
asm mov cx, startScanOffset

asm mov dx,0x3d4 ; 3d4h is the CRTC register

asm mov al,0x0c ; Tell the CRTC we want to update
asm out dx,al ; the start address high register
asm inc dx
asm mov al,ch
asm out dx,al ; set the high byte
asm dec dx
asm mov al,0x0d ; Tell the CRTC we want to update
asm out dx,al ; the start address low register
asm inc dx
asm mov al,cl
asm out dx,al ; set the low byte
```

This code looks like it would work, but there is a major flaw with it. If you were to run it, every once in a while the expected screen shown below...



...would instead appear distorted:



This glitch shows both misalignment and parts of two pages. This problem has to do with atomicity. The CRTC starting address is a 16-bit value but the out instruction can only write 8 bits at a time. If the pages are setup one after another like this.

0x0000	0x3E80	0x7000	
Page 1	Page 2	Page 3	

In VRAM, Page 1 is at 0x0000, page 2 at 0x3E80 and page 3 at 0x7000. Instructing the CRTC to use page 2 instead of page 1 requires updating the high byte 0x00 to 0x3E and the low byte 0x00 to 0x80. Since updates are not atomic, poor timing could result in the CRTC picking up a value of 0x3E00 instead of 0x3E80:

```

asm  mov  cx, startScanOffset

asm  mov  dx,0x3d4    ; 3d4h  is the CRTC register

asm  mov  al,0xc      ; Tell the CRTC we want to update
asm  out  dx,al       ; the start address high register
asm  inc  dx
asm  mov  al,ch
asm  out  dx,al       ; set the high byte

;***** CRTC SCAN START HERE !!!!!!!! *****
;***** AND SHOWS 2 PARTIAL FRAMEBUFFERS *****

asm  mov  al,0xd      ; Tell the CRTC we want to update
asm  out  dx,al       ; start address low register
asm  inc  dx
asm  mov  al,cl
asm  out  dx,al       ; set the low byte

```

How do you update atomically a 2-byte value with a 1-byte operation? Take a look at how Wolfenstein sets up its pages:

```

#define SCREENBWIDE    80
...
#define SCREENSIZE    (SCREENBWIDE*208)
#define PAGE1START    0
#define PAGE2START    (SCREENSIZE)
#define PAGE3START    (SCREENSIZE*2u)
#define FREESTART     (SCREENSIZE*3u)

```

Notice how it uses a value of 208 for the height of a framebuffer. At first, this doesn't make sense as the screen is 200 pixels tall. I thought it was a typo (after all 0 and 8 are visually close) but this was done intentionally. The trick here is to use a little bit of padding after each page so the addresses only differ by their high byte value.

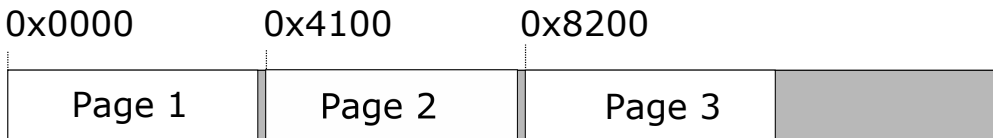


Figure 4.16: A little padding between pages makes all starting address a multiple of 256.

Now page 0 is at 0x0000, page 1 at 0x4100 and page 2 at 0x8200. Moving from any page to another requires updating only the high 8 bits, which makes flipping buffer an atomic operation.

4.5.3 Profound Carnage

After the signon screen comes the "rating" screen. There were no official ratings for video games in 1991 since the ESRB⁹ would not be established until 1994 in response to criticisms of excessive violence (a.k.a Doom) or sexual content. Yet the game still displayed a warning, another occasion for id to show its irreverence.

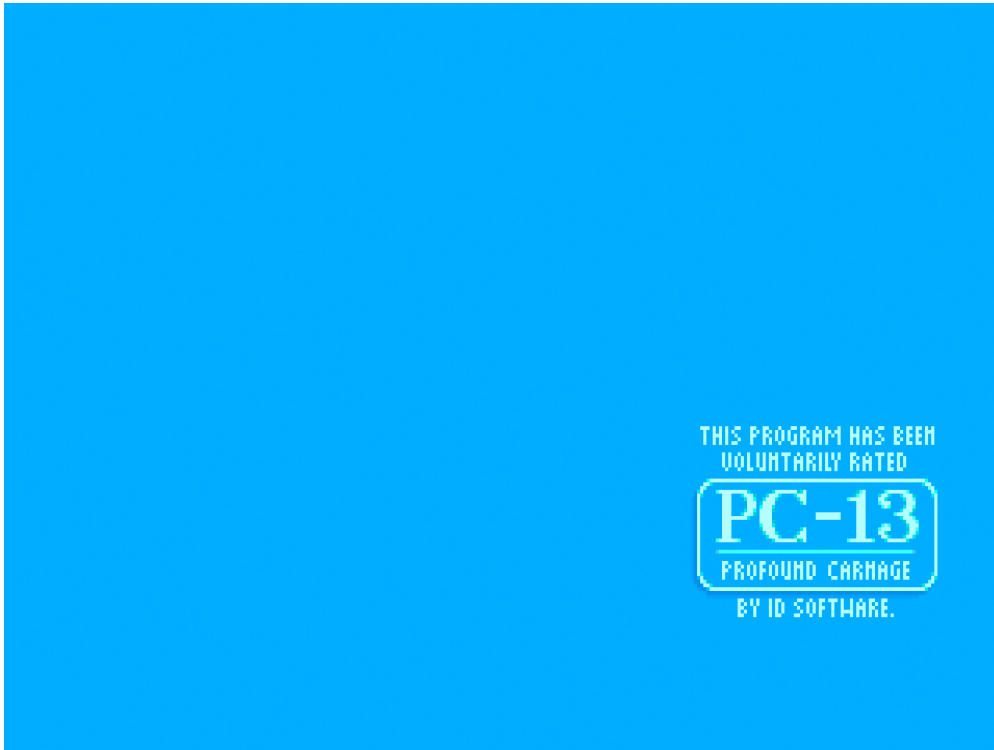


Figure 4.17: This program has been voluntarily rated PC "Profound Carnage"-13.

The PC-13 rating screen was of course inspired by the official Motion Picture Association of America film rating system PG-13 rating logo.

⁹Entertainment Software Rating Board, the organization in charge of assigning age and content ratings.